

TUGAS BESAR IF3170
TEORI BAHASA FORMAL DAN OTOMATA
MILESTONE 3 - SEMANTIC ANALYSIS



Muhammad Ra'if Alkautsar	13523011
Fajar Kurniawan	13523076
Darrel Adinarya Sunanda	13523061
Reza Ahmad Syarif	13523119

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi.....	2
1. Landasan Teori.....	3
a. Semantic Analysis.....	3
b. Abstract Syntax Tree.....	4
2. Perancangan & Implementasi Program.....	6
a. Teknologi Yang Digunakan.....	6
b. Struktur Program.....	7
c. Fungsi/Kelas Utama.....	8
d. Alur Kerja Program.....	26
3. Pengujian.....	28
a. dasar.pas.....	28
b. test_var_param.pas.....	32
c. test_constants.pas.....	48
d. error_type_redeclaration.pas.....	55
e. error_for_undeclared.pas.....	57
4. Kesimpulan & Saran.....	60
a. Kesimpulan.....	60
b. Saran.....	61
Lampiran.....	62
Referensi.....	63

1. Landasan Teori

Dalam pemrosesan bahasa pemrograman, setelah tahap analisis sintaksis (*syntax analysis*) oleh parser, parse tree yang dihasilkan akan diubah menjadi Abstract Syntax Tree.

a. Semantic Analysis

Semantic Analysis adalah bagian ketiga dari compiler yang memastikan declaration dan statement pada program bersifat *semantically correct*. Semantic Analyzer menggunakan syntax tree dan symbol table untuk mengecek apakah program konsisten secara semantik dengan definisi bahasa/*language* dari program tersebut. Ia mengumpulkan informasi dari type dan menyimpannya di dalam syntax tree atau symbol table. *Type information* ini selanjutnya digunakan compiler pada tahap intermediate-code generation.

Dengan kata lain, semantic analyzer adalah bagian dari compiler yang mengecek bahwa source code masuk akal secara logika. Ia memastikan bahwa operasi yang dilakukan di kode benar secara logika. Bagian ini memastikan source code mengikuti aturan dari bahasa pemrograman dari sisi logik dan penggunaan data.

Semantic Errors atau kesalahan yang dapat dikenali oleh semantic analyzer adalah type mismatch, undeclared variables, reserved identifier misuse, Beberapa fungsi utama dari Semantic Analysis adalah untuk melakukan type checking, yaitu memastikan bahwa data type yang digunakan konsisten dengan pendefinisian type tersebut. Kemudian fungsi scope checking, yaitu memastikan variable, function, dan symbol lainnya digunakan dalam scope di mana mereka didefinisikan. Fungsi ketiga adalah flow control check, yaitu menjaga penggunaan struktur yang benar (misal tidak boleh ada break statement di luar loop). Fungsi lainnya meliputi identifier checking, function/procedure call checking, symbol table management, dsb.

Semantic Analyzer dapat mengecek undefined variables, duplicate identifier, function overloading, dan sebagainya. Untuk bisa melakukan hal itu, analyzer menggunakan sebuah symbol table, yaitu suatu struktur data yang diisi pada fase syntax analysis untuk mencatat semua identifier dan propertinya seperti tipe, scope, lokasi di memory, dan informasi lain yang dibutuhkan untuk validasi/analisis.

Contoh hal yang dilakukan semantic analyzer adalah sebagai berikut. Misalkan pada kode:

```
float x = 10.1;  
float y = x*30;
```

Integer 30 akan di-typecast menjadi float 30.0 sebelum perkalian. Namun, contoh selanjutnya berbeda:

```
int a = 5;
```

```
float b = 3.5;  
a = a + b;
```

Dari kode tersebut, dilakukan type checking. Variabel a adalah int dan b adalah float. Penjumlahan keduanya menghasilkan float dan tidak bisa disimpan dalam variabel a yang merupakan int sehingga muncul error “Type mismatch: cannot assign float to int”.

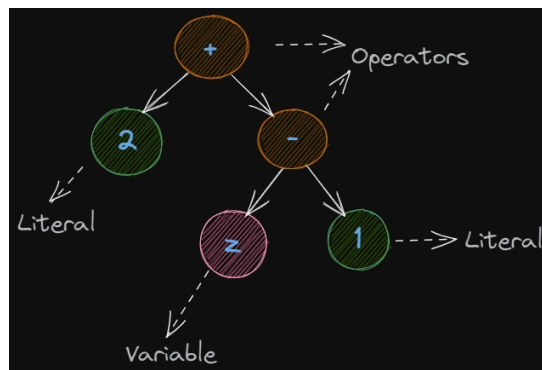
Terdapat dua jenis semantic analysis. Static semantics analysis mengecek semantik pada waktu kompilasi. Sedangkan dynamic semantic analysis mengecek semantik pada runtime.

b. Abstract Syntax Tree

Pada milestone sebelumnya, dibuat parse tree. Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node di dalam parse tree merepresentasikan produksi grammar, sehingga pohon ini memuat semua simbol terminal dan non-terminal. Sedangkan Abstract Syntax Tree (AST) menghilangkan detail-detail tidak penting yang tidak dibutuhkan compiler (seperti titik koma, tanda kurung, dsb. tergantung implementasi) dan meninggalkan hanya informasi-informasi yang diperlukan compiler untuk memahami struktur kode. Maka dari itu, AST adalah tree yang merepresentasikan struktur sintaksis dari sebuah source code. Tidak ada representasi AST yang umum dan implementasi aktualnya akan berbeda dari bahasa ke bahasa.

Secara umum, pada AST, hubungan antar token di source code direpresentasikan oleh simpul/*node* dan anak-anak dari simpul tersebut. Setiap node mengandung informasi mengenai tipe token dan informasi mengenai token serta data yang berhubungan. Misalnya jika sebuah simpul merepresentasikan sebuah pemanggilan fungsi maka anak-anaknya adalah argument dari pemanggilan fungsi tersebut.

Sebagai contoh $2+(z-1)$ dapat direpresentasikan sebagai pohon seperti berikut:



Sumber: <https://dev.to/balapriya/abstract-syntax-tree-ast-explained-in-plain-english-1h38>

Pada contoh di atas, simbol $+$ dan $-$ adalah operator dan z sebuah variabel, 1 dan 2 hanyalah literal biasa. Tanda kurung dihilangkan karena tidak diperlukan dan tidak relevan untuk compiler.

2. Perancangan & Implementasi Program

a. Teknologi Yang Digunakan

Implementasi semantic analyzer ini memanfaatkan tiga teknologi utama dari C++17 yaitu smart pointers, standard library containers, dan visitor pattern.

1. Smart Pointers untuk Manajemen Memori

Smart pointers, khususnya `shared_ptr`, digunakan untuk manajemen memori otomatis pada struktur Abstract Syntax Tree (AST). Setiap `ASTNode` menggunakan `shared_ptr` untuk menyimpan referensi ke child nodes seperti declarations, expressions, dan statements. Pemilihan `shared_ptr` didasarkan pada kebutuhan shared ownership yaitu satu node dapat direferensikan oleh parent node, disimpan dalam collections, atau diakses dari berbagai tahap analisis (AST building, semantic checking). Smart pointer menyediakan reference counting otomatis yang memastikan node akan di-dealokasi hanya ketika tidak ada lagi referensi yang menggunakannya, sehingga dapat mengeliminasi risiko memory leak dan dangling pointer.

2. Standard Library Containers

Standard library containers berupa `std::vector` dan `std::map` digunakan dalam konteks krusial:

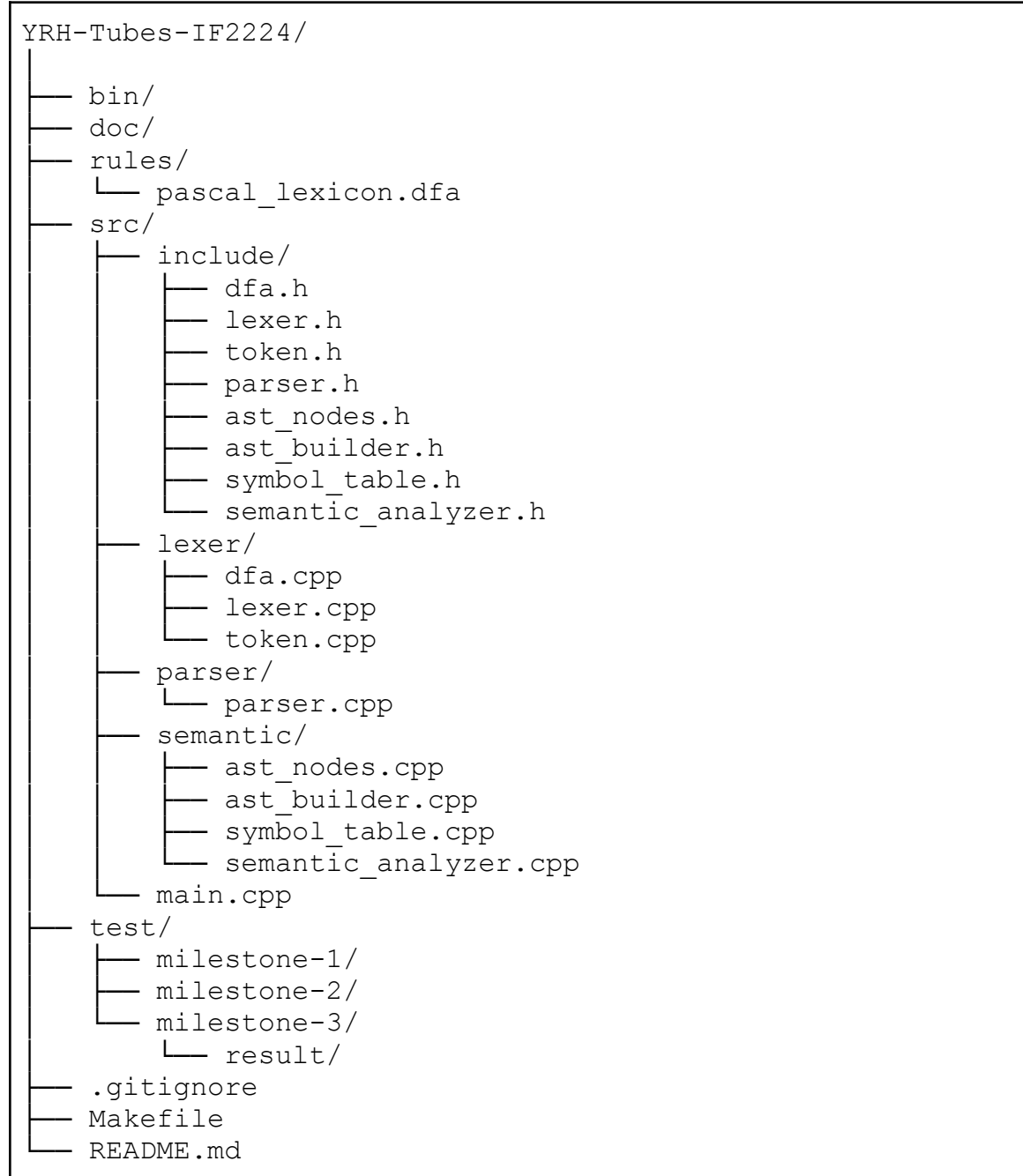
- `vector<shared_ptr<ASTNode>>` untuk menyimpan children nodes dalam AST seperti statement list dan declaration list. Vector dipilih karena menyediakan $O(1)$ random access dan dynamic sizing yang fleksibel.
- `map<string, int>` untuk menyimpan nilai konstanta (`constantValues` di `ASTBuilder`) yang diperlukan untuk evaluasi array bounds seperti larik `[1..ten]`. Map menyediakan $O(\log n)$ lookup yang efisien berdasarkan nama konstanta.
- `vector<TabEntry>`, `vector<BtabEntry>`, `vector<AtabEntry>` untuk symbol tables. Vector dipilih karena Pascal menggunakan index-based referencing dimana setiap identifier memiliki TAB index tetap, sehingga $O(1)$ random access sangat penting untuk lookup efisien.

3. Visitor Pattern

Visitor pattern diimplementasikan melalui class `SemanticAnalyzer` yang memiliki method `visit*()` untuk setiap jenis node (`visitProgram`, `visitVarDecl`, `visitAssign`, dll.). Pattern ini memisahkan algoritma traversal tree dari struktur node, memungkinkan penambahan operasi baru tanpa memodifikasi class hierarchy AST. Setiap node memanggil `accept(visitor)` yang men-dispatch ke method `visit*()` sesuai tipe konkret node tersebut. Inheritance hierarchy digunakan dimana `ASTNode` adalah base class dengan virtual methods, dan derived classes seperti `ProgramNode`, `VarDeclNode`, `BinOpNode` meng-override untuk behavior spesifik, mengaktifkan polymorphism untuk menyimpan berbagai jenis node dalam satu container.

b. Struktur Program

Berikut adalah struktur direktori dan file dari program Syntax Analyzer ini.



Berikut adalah penjelasan dari struktur tersebut.

1. Header Files (include)
 - a. token.h: Mendefinisikan enum TokenType dan class Token untuk representasi token
 - b. dfa.h: Mendefinisikan DFA untuk pattern matching dalam lexer
 - c. lexer.h: Mendefinisikan Lexer class untuk lexical analysis

- d. parser.h: Mendefinisikan Parser dan ParseNode class untuk syntax analysis
 - e. ast_nodes.h: Mendefinisikan hierarchy AST nodes (ASTNode, ProgramNode, VarDeclNode, dll.)
 - f. ast_builder.h: Mendefinisikan ASTBuilder class untuk konversi parse tree ke AST
 - g. symbol_table.h: Mendefinisikan SymbolTable dan struktur TAB, BTAB, ATAB
 - h. semantic_analyzer.h: Mendefinisikan SemanticAnalyzer class dengan visitor pattern
2. Implementation Files (src)
- a. lexer/Lexical Analysis
 - i. dfa.cpp: Implementasi DFA state machine
 - ii. lexer.cpp: Implementasi tokenization process
 - iii. token.cpp: Implementasi Token class
 - b. parser/Syntax Analysis
 - i. parser.cpp: Implementasi parsing dengan recursive descent
 - c. semantic/Semantic Analysis
 - i. ast_nodes.cpp: Implementasi AST node classes dan visitor methods
 - ii. ast_builder.cpp: Implementasi konversi parse tree ke AST
 - iii. symbol_table.cpp: Implementasi symbol table management (TAB, BTAB, ATAB)
 - iv. semantic_analyzer.cpp: Implementasi semantic checking dan type analysis
3. Resources
- a. pascal_lexicon.dfa: definisi aturan DFA
 - b. milestone-1: Test cases untuk lexical analyzer
 - c. milestone-2: Test cases untuk syntax analyzer
 - d. milestone-3/: Test cases untuk semantic analyzer

c. Fungsi/Kelas Utama

Berikut adalah kelas utama dan beberapa fungsi didalamnya yang berperan pada proses syntax analysis.

1. Kelas ASTNode

ASTNode adalah representasi abstrak dari elemen program setelah parsing. ASTNode berfungsi menyimpan struktur sintaks yang disederhanakan beserta metadata yang diperlukan untuk analisis semantik seperti tipe data, indeks simbol pada symbol table, level scope, dan nomor baris. Setiap turunan ASTNode (misalnya ProgramNode, VarDeclNode, AssignNode, BinOpNode, dan

NumberNode) mengimplementasikan metode akses dan perekaman metadata serta pola visitor (accept) untuk traversal.

ASTNode juga menyediakan fungsi cetak/debug yang menampilkan struktur pohon dan informasi dekorasi sehingga hasil analisis (decorated AST) mudah dibaca. Berbeda dengan parse tree, ASTNode hanya mengandung informasi yang esensial untuk kompilasi program nantinya.

```
#ifndef AST_NODES_H
#define AST_NODES_H

#include <string>
#include <vector>
#include <memory>

using namespace std;
class ASTVisitor;

enum class DataType { VOID, INTEGER, REAL, BOOLEAN, CHAR, STRING, ARRAY, RECORD, UNKNOWN };

enum class ObjectType {
    CONSTANT,
    VARIABLE,
    TYPE_DEF,
    PROCEDURE,
    FUNCTION,
    PARAMETER,
    ARRAY_TYPE,
    RECORD_TYPE,
    PROGRAM
};

class ASTNode {
protected:
    DataType dataType;      // Tipe data dari node
    int symbolTableIndex;   // Index di symbol table (-1 jika tidak ada)
    int scopeLevel;        // Lexical level/scope
    int lineNumber;        // Line number untuk error reporting
    int columnNumber;      // Column number untuk error reporting

public:
    ASTNode();
    virtual ~ASTNode() = default;

    // Getters
    DataType getDataType() const {
        return dataType;
    }
    int getSymbolTableIndex() const {
        return symbolTableIndex;
    }
    int getScopeLevel() const {
        return scopeLevel;
    }
    int getLineNumber() const {
        return lineNumber;
    }
    int getColumnNumber() const {
        return columnNumber;
    }

    // Setters untuk semantic analysis
    void setDataType(DataType type) {
        dataType = type;
    }
};
```

```

    }
    void setSymbolTableIndex(int index) {
        symbolTableIndex = index;
    }
    void setScopeLevel(int level) {
        scopeLevel = level;
    }
    void setLineNumber(int line) {
        lineNumber = line;
    }
    void setColumnNumber(int column) {
        columnNumber = column;
    }
}

// Virtual method untuk visitor pattern
virtual void accept(ASTVisitor* visitor) = 0;

// Virtual method untuk printing AST
virtual void print(int indent = 0) const = 0;
};

// ===== PROGRAM NODE =====
class ProgramNode : public ASTNode {
private:
    string name;
    vector<shared_ptr<ASTNode>> declarations;
    shared_ptr<ASTNode> compoundStatement;

public:
    ProgramNode(const string& programName);

    void addDeclaration(shared_ptr<ASTNode> decl);
    void setCompoundStatement(shared_ptr<ASTNode> stmt);

    string getName() const {
        return name;
    }
    const vector<shared_ptr<ASTNode>>& getDeclarations() const {
        return declarations;
    }
    shared_ptr<ASTNode> getCompoundStatement() const {
        return compoundStatement;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// ===== DECLARATION NODES =====

// Variable Declaration Node
class VarDeclNode : public ASTNode {
private:
    string name;
    DataType varType;
    string customTypeName; // For custom types (e.g., "DaftarNilai")
    bool isParameter;      // true jika ini adalah parameter
    bool isVarParameter;   // true jika parameter by reference (var)

public:
    VarDeclNode(const string& varName, DataType type);

    string getName() const {
        return name;
    }
    DataType getVarType() const {
        return varType;
    }
}

```

```

    string getCustomTypeName() const {
        return customTypeName;
    }
    bool getIsParameter() const {
        return isParameter;
    }
    bool getIsVarParameter() const {
        return isVarParameter;
    }

    void setCustomTypeName(const string& typeName) {
        customTypeName = typeName;
    }
    void setIsParameter(bool param) {
        isParameter = param;
    }
    void setIsVarParameter(bool varParam) {
        isVarParameter = varParam;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Constant Declaration Node
class ConstDeclNode : public ASTNode {
private:
    string name;
    shared_ptr<ASTNode> value; // Bisa NumberNode, StringNode, BoolNode

public:
    ConstDeclNode(const string& constName, shared_ptr<ASTNode> val);

    string getName() const {
        return name;
    }
    shared_ptr<ASTNode> getValue() const {
        return value;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Type Declaration Node (untuk array dan record)
class TypeDeclNode : public ASTNode {
private:
    string name;
    DataType typeKind; // ARRAY atau RECORD
    shared_ptr<ASTNode> typeDefinition; // ArrayTypeNode atau RecordTypeNode

public:
    TypeDeclNode(const string& typeName, DataType kind, shared_ptr<ASTNode> typeDef);

    string getName() const {
        return name;
    }
    DataType getTypeKind() const {
        return typeKind;
    }
    shared_ptr<ASTNode> getTypeDefinition() const {
        return typeDefinition;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

```

```

// Array Type Node
class ArrayTypeNode : public ASTNode {
private:
    int lowBound;
    int highBound;
    DataType elementType;
    shared_ptr<ArrayTypeNode> nestedElementType; // For multi-dimensional arrays
    int arrayTableIndex; // Index di atab

public:
    ArrayTypeNode(int low, int high, DataType elemType);
    ArrayTypeNode(int low, int high, shared_ptr<ArrayTypeNode> nestedElemType);

    int getLowBound() const {
        return lowBound;
    }
    int getHighBound() const {
        return highBound;
    }
    DataType getElementType() const {
        return elementType;
    }
    shared_ptr<ArrayTypeNode> getNestedElementType() const {
        return nestedElementType;
    }
    bool isMultiDimensional() const {
        return nestedElementType != nullptr;
    }
    int getArrayTableIndex() const {
        return arrayTableIndex;
    }

    void setArrayTableIndex(int index) {
        arrayTableIndex = index;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

```

```

// Record Type Node
class RecordTypeNode : public ASTNode {
private:
    vector<shared_ptr<VarDeclNode>> fields;
    int blockTableIndex; // Index di btab

public:
    RecordTypeNode();

    void addField(shared_ptr<VarDeclNode> field);
    const vector<shared_ptr<VarDeclNode>>& getFields() const {
        return fields;
    }
    int getBlockTableIndex() const {
        return blockTableIndex;
    }

    void setBlockTableIndex(int index) {
        blockTableIndex = index;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

```

```

// Procedure Declaration Node
class ProcedureDeclNode : public ASTNode {
private:

```

```

    string name;
    vector<shared_ptr<VarDeclNode>> parameters;
    vector<shared_ptr<ASTNode>> declarations;
    shared_ptr<ASTNode> compoundStatement;
    int blockTableIndex; // Index di btab

public:
    ProcedureDeclNode(const string& procName);

    void addParameter(shared_ptr<VarDeclNode> param);
    void addDeclaration(shared_ptr<ASTNode> decl);
    void setCompoundStatement(shared_ptr<ASTNode> stmt);

    string getName() const {
        return name;
    }
    const vector<shared_ptr<VarDeclNode>>& getParameters() const {
        return parameters;
    }
    const vector<shared_ptr<ASTNode>>& getDeclarations() const {
        return declarations;
    }
    shared_ptr<ASTNode> getCompoundStatement() const {
        return compoundStatement;
    }
    int getBlockTableIndex() const {
        return blockTableIndex;
    }

    void setBlockTableIndex(int index) {
        blockTableIndex = index;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Function Declaration Node
class FunctionDeclNode : public ASTNode {
private:
    string name;
    DataType returnType;
    vector<shared_ptr<VarDeclNode>> parameters;
    vector<shared_ptr<ASTNode>> declarations;
    shared_ptr<ASTNode> compoundStatement;
    int blockTableIndex; // Index di btab

public:
    FunctionDeclNode(const string& funcName, DataType retType);

    void addParameter(shared_ptr<VarDeclNode> param);
    void addDeclaration(shared_ptr<ASTNode> decl);
    void setCompoundStatement(shared_ptr<ASTNode> stmt);

    string getName() const {
        return name;
    }
    DataType getReturnType() const {
        return returnType;
    }
    const vector<shared_ptr<VarDeclNode>>& getParameters() const {
        return parameters;
    }
    const vector<shared_ptr<ASTNode>>& getDeclarations() const {
        return declarations;
    }
    shared_ptr<ASTNode> getCompoundStatement() const {
        return compoundStatement;
    }
};

```

```

    }
    int getBlockTableIndex() const {
        return blockTableIndex;
    }

    void setBlockTableIndex(int index) {
        blockTableIndex = index;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// ===== STATEMENT NODES =====

// Compound Statement Node (begin...end)
class CompoundStatementNode : public ASTNode {
private:
    vector<shared_ptr<ASTNode>> statements;

public:
    CompoundStatementNode();

    void addStatement(shared_ptr<ASTNode> stmt);
    const vector<shared_ptr<ASTNode>>& getStatements() const {
        return statements;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Assignment Statement Node
class AssignNode : public ASTNode {
private:
    shared_ptr<ASTNode> target; // VarNode atau ArrayAccessNode atau RecordAccessNode
    shared_ptr<ASTNode> value; // Expression

public:
    AssignNode(shared_ptr<ASTNode> lhs, shared_ptr<ASTNode> rhs);

    shared_ptr<ASTNode> getTarget() const {
        return target;
    }
    shared_ptr<ASTNode> getValue() const {
        return value;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// If Statement Node
class IfNode : public ASTNode {
private:
    shared_ptr<ASTNode> condition;
    shared_ptr<ASTNode> thenStatement;
    shared_ptr<ASTNode> elseStatement; // nullptr jika tidak ada else

public:
    IfNode(shared_ptr<ASTNode> cond, shared_ptr<ASTNode> thenStmt,
           shared_ptr<ASTNode> elseStmt = nullptr);

    shared_ptr<ASTNode> getCondition() const {
        return condition;
    }
    shared_ptr<ASTNode> getThenStatement() const {
        return thenStatement;
    }

```

```

    }
    shared_ptr<ASTNode> getElseStatement() const {
        return elseStatement;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// While Statement Node
class WhileNode : public ASTNode {
private:
    shared_ptr<ASTNode> condition;
    shared_ptr<ASTNode> body;

public:
    WhileNode(shared_ptr<ASTNode> cond, shared_ptr<ASTNode> bodyStmt);

    shared_ptr<ASTNode> getCondition() const {
        return condition;
    }
    shared_ptr<ASTNode> getBody() const {
        return body;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// For Statement Node
class ForNode : public ASTNode {
private:
    string loopVariable;
    shared_ptr<ASTNode> startValue;
    shared_ptr<ASTNode> endValue;
    bool isDownto; // true jika downto, false jika to
    shared_ptr<ASTNode> body;

public:
    ForNode(const string& loopVar, shared_ptr<ASTNode> start, shared_ptr<ASTNode> end, bool downto,
            shared_ptr<ASTNode> bodyStmt);

    string getLoopVariable() const {
        return loopVariable;
    }
    shared_ptr<ASTNode> getStartValue() const {
        return startValue;
    }
    shared_ptr<ASTNode> getEndValue() const {
        return endValue;
    }
    bool getIsDownto() const {
        return isDownto;
    }
    shared_ptr<ASTNode> getBody() const {
        return body;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Procedure/Function Call Node
class ProcCallNode : public ASTNode {
private:
    string name;
    vector<shared_ptr<ASTNode>> arguments;

```

```

public:
    ProcCallNode(const string& procName);

    void addArgument(shared_ptr<ASTNode> arg);

    string getName() const {
        return name;
    }
    const vector<shared_ptr<ASTNode>>& getArguments() const {
        return arguments;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// ===== EXPRESSION NODES =====

// Binary Operation Node
class BinOpNode : public ASTNode {
private:
    string op; // +, -, *, /, div, mod, and, or, =, <>, <, >, <=, >=
    shared_ptr<ASTNode> left;
    shared_ptr<ASTNode> right;

public:
    BinOpNode(const string& operation, shared_ptr<ASTNode> leftOperand,
              shared_ptr<ASTNode> rightOperand);

    string getOp() const {
        return op;
    }
    shared_ptr<ASTNode> getLeft() const {
        return left;
    }
    shared_ptr<ASTNode> getRight() const {
        return right;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Unary Operation Node
class UnaryOpNode : public ASTNode {
private:
    string op; // -, not
    shared_ptr<ASTNode> operand;

public:
    UnaryOpNode(const string& operation, shared_ptr<ASTNode> operandNode);

    string getOp() const {
        return op;
    }
    shared_ptr<ASTNode> getOperand() const {
        return operand;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Number Literal Node
class NumberNode : public ASTNode {
private:
    int intValue;
    double realValue;

```



```

    bool isReal;

public:
    NumberNode(int value);
    NumberNode(double value);

    int getIntValue() const {
        return intValue;
    }
    double getRealValue() const {
        return realValue;
    }
    bool getIsReal() const {
        return isReal;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// String Literal Node
class StringNode : public ASTNode {
private:
    string value;

public:
    StringNode(const string& str);

    string getValue() const {
        return value;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Char Literal Node
class CharNode : public ASTNode {
private:
    char value;

public:
    CharNode(char ch);

    char getValue() const {
        return value;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Boolean Literal Node
class BoolNode : public ASTNode {
private:
    bool value;

public:
    BoolNode(bool val);

    bool getValue() const {
        return value;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

```

```

// Variable Reference Node
class VarNode : public ASTNode {
private:
    string name;

public:
    VarNode(const string& varName);

    string getName() const {
        return name;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Array Access Node (arr[index])
class ArrayAccessNode : public ASTNode {
private:
    shared_ptr<ASTNode> arrayVar; // VarNode atau nested ArrayAccessNode
    shared_ptr<ASTNode> index;    // Expression

public:
    ArrayAccessNode(shared_ptr<ASTNode> arr, shared_ptr<ASTNode> idx);

    shared_ptr<ASTNode> getArrayVar() const {
        return arrayVar;
    }
    shared_ptr<ASTNode> getIndex() const {
        return index;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// Record Access Node (record.field)
class RecordAccessNode : public ASTNode {
private:
    shared_ptr<ASTNode> recordVar; // VarNode atau nested RecordAccessNode
    string fieldName;

public:
    RecordAccessNode(shared_ptr<ASTNode> rec, const string& field);

    shared_ptr<ASTNode> getRecordVar() const {
        return recordVar;
    }
    string getFieldName() const {
        return fieldName;
    }

    void accept(ASTVisitor* visitor) override;
    void print(int indent = 0) const override;
};

// ===== VISITOR INTERFACE =====

// Abstract Visitor class untuk visitor pattern
class ASTVisitor {
public:
    virtual ~ASTVisitor() = default;

    // Visit methods untuk setiap node type
    virtual void visitProgram(ProgramNode* node) = 0;
    virtual void visitVarDecl(VarDeclNode* node) = 0;
    virtual void visitConstDecl(ConstDeclNode* node) = 0;
    virtual void visitTypeDecl(TypeDeclNode* node) = 0;
};

```

```

virtual void visitArrayType(ArrayTypeNode* node) = 0;
virtual void visitRecordType(RecordTypeNode* node) = 0;
virtual void visitProcedureDecl(ProcedureDeclNode* node) = 0;
virtual void visitFunctionDecl(FunctionDeclNode* node) = 0;
virtual void visitCompoundStatement(CompoundStatementNode* node) = 0;
virtual void visitAssign(AssignNode* node) = 0;
virtual void visitIf(IfNode* node) = 0;
virtual void visitWhile(WhileNode* node) = 0;
virtual void visitFor(ForNode* node) = 0;
virtual void visitProcCall(ProcCallNode* node) = 0;
virtual void visitBinOp(BinOpNode* node) = 0;
virtual void visitUnaryOp(UnaryOpNode* node) = 0;
virtual void visitNumber(NumberNode* node) = 0;
virtual void visitString(StringNode* node) = 0;
virtual void visitChar(CharNode* node) = 0;
virtual void visitBool(BoolNode* node) = 0;
virtual void visitVar(VarNode* node) = 0;
virtual void visitArrayAccess(ArrayAccessNode* node) = 0;
virtual void visitRecordAccess(RecordAccessNode* node) = 0;
};

// ===== UTILITY FUNCTIONS =====

// Helper function untuk convert DataType ke string
string dataTypeToString(DataType type);

// Helper function untuk convert ObjectType ke string
string objectTypeToString(ObjectType type);

#endif // AST_NODES_H

```

2. Kelas ASTBuilder

ASTBuilder adalah kelas yang bertanggung jawab mengonversi parse tree (ParseNode) menjadi AST yang sifatnya lebih ringkas sehingga mudah untuk dilakukan analisis semantik dan pembuatan kode. Tugas utama dari ASTBuilder antara lain meliputi pemetaan aturan grammar menjadi node AST yang sesuai (program, deklarasi, statement, ekspresi), parsing literal (angka, string, char, boolean), serta transfer posisi (line number) dari parse node ke AST node melalui helper seperti setPosition. ASTBuilder juga harus menangani pengelompokan deklarasi (multiple identifier-list : type ;), mengekstrak batas untuk subrange/larik dengan penanganan aman terhadap stoi/stoll/stod (try/catch), dan menemukan token operator langsung (RELATIONAL / ARITHMETIC / LOGICAL) ketika membentuk BinOp/UnaryOp.

```

#ifndef AST_BUILDER_H
#define AST_BUILDER_H

#include "ast_nodes.h"
#include "parser.h"
#include <memory>
#include <vector>
#include <string>

using namespace std;

```

```

/**
 * AST Builder - Converts Parse Tree to Abstract Syntax Tree
 *
 * This class traverses the parse tree produced by the Parser and
 * constructs a simplified AST suitable for semantic analysis.
 */
class ASTBuilder {
private:
    shared_ptr<ParseNode> parseTree;
    vector<string> errors;

    // Helper functions
    bool isTerminal(shared_ptr<ParseNode> node) const;
    string getTokenValue(shared_ptr<ParseNode> node) const;
    string getNodeTypes(shared_ptr<ParseNode> node) const;
    shared_ptr<ParseNode> findChild(shared_ptr<ParseNode> node, const string& type) const;
    vector<shared_ptr<ParseNode>> findChildren(shared_ptr<ParseNode> node,
                                             const string& type) const;
    void setPosition(shared_ptr<ASTNode> astNode, shared_ptr<ParseNode> parseNode);

    // Type conversion
    DataType stringToDataType(const string& typeStr);

    // Main conversion functions
    shared_ptr<ProgramNode> convertProgram(shared_ptr<ParseNode> node);

    // Declaration conversions
    vector<shared_ptr<ASTNode>> convertDeclarationPart(shared_ptr<ParseNode> node);
    vector<shared_ptr<VarDeclNode>> convertVarDeclaration(shared_ptr<ParseNode> node);
    vector<shared_ptr<ConstDeclNode>> convertConstDeclaration(shared_ptr<ParseNode> node);
    vector<shared_ptr<TypeDeclNode>> convertTypeDeclaration(shared_ptr<ParseNode> node);
    shared_ptr<ProcedureDeclNode> convertProcedureDeclaration(shared_ptr<ParseNode> node);
    shared_ptr<FunctionDeclNode> convertFunctionDeclaration(shared_ptr<ParseNode> node);

    // Type conversions
    DataType convertType(shared_ptr<ParseNode> node);
    shared_ptr<ArrayTypeNode> convertArrayType(shared_ptr<ParseNode> node);
    shared_ptr<RecordTypeNode> convertRecordType(shared_ptr<ParseNode> node);

    // Parameter conversions
    vector<shared_ptr<VarDeclNode>> convertFormalParameterList(shared_ptr<ParseNode> node);

    // Statement conversions
    shared_ptr<CompoundStatementNode> convertCompoundStatement(shared_ptr<ParseNode> node);
    vector<shared_ptr<ASTNode>> convertStatementList(shared_ptr<ParseNode> node);
    shared_ptr<ASTNode> convertStatement(shared_ptr<ParseNode> node);
    shared_ptr<AssignNode> convertAssignmentStatement(shared_ptr<ParseNode> node);
    shared_ptr<IfNode> convertIfStatement(shared_ptr<ParseNode> node);
    shared_ptr<WhileNode> convertWhileStatement(shared_ptr<ParseNode> node);
    shared_ptr<ForNode> convertForStatement(shared_ptr<ParseNode> node);
    shared_ptr<ProcCallNode> convertProcedureCall(shared_ptr<ParseNode> node);

    // Expression conversions
    shared_ptr<ASTNode> convertExpression(shared_ptr<ParseNode> node);
    shared_ptr<ASTNode> convertSimpleExpression(shared_ptr<ParseNode> node);
    shared_ptr<ASTNode> convertTerm(shared_ptr<ParseNode> node);
    shared_ptr<ASTNode> convertFactor(shared_ptr<ParseNode> node);

    // Helper for variable reference (could be array access or record access)
    shared_ptr<ASTNode> convertVariableAccess(shared_ptr<ParseNode> node);

    // Helper to extract number from nested expression tree
    int extractNumberFromExpression(shared_ptr<ParseNode> expr) const;

    // Error handling
    void error(const string& message);

public:

```

```

ASTBuilder();

// Main build function
shared_ptr<ProgramNode> buildAST(shared_ptr<ParseNode> parseTree);

// Error handling
bool hasErrors() const {
    return !errors.empty();
}
const vector<string>& getErrors() const {
    return errors;
}
};

#endif // AST_BUILDER_H

```

3. Kelas SymbolTable

SymbolTable adalah kelas yang menyimpan seluruh informasi identifier dan tipe yang diperlukan sepanjang proses kompilasi, yaitu tabel identifier (tab), tabel blok/scope (btabs), dan tabel metadata array (atab).

Struktur internal menyediakan operasi masuk/keluar scope, penambahan simbol (addSymbol), pencarian cepat berdasarkan nama (lookupSymbol) dengan opsi hanya scope saat ini, serta utilitas untuk mengambil/ubah entry dan mencetak tabel untuk debugging. Setiap TabEntry mencatat atribut seperti nama, link, kelas objek (VAR/TYPE/PROC/FUNC), kode tipe, level scope, dan alamat. SemanticAnalyzer mengandalkan SymbolTable untuk validasi deklarasi/penamaan, penentuan tipe, serta dekorasi AST nodes dengan indeks tab dan level scope.

```

#ifndef SYMBOL_TABLE_H
#define SYMBOL_TABLE_H

#include <string>
#include <vector>
#include <stack>
#include <unordered_map>
#include <iostream>

using namespace std;

enum class ObjectClass {
    CONSTANT = 0,
    VARIABLE = 1,
    TYPE = 2,
    PROCEDURE = 3,
    FUNCTION = 4,
    RESERVED = 5,
    PROGRAM = 6
};

enum class TypeCode {
    NOTYP = 0,
    INTS = 1,
    REALS = 2,

```

```

BOOLS = 3,
CHARS = 4,
ARRAYS = 5,
RECORDS = 6
};

struct TabEntry {
    string identifier; // Nama identifier
    int link;          // Link ke identifier sebelumnya di block yang sama
    int obj;           // Object class (constant, variable, type, procedure, function)
    int typ;           // Type code
    int ref;           // Reference ke btab/atab (untuk composite types)
    int nrm;           // Normal: 1 = normal variable, 0 = var parameter (by reference)
    int lev;           // Lexical level (0 = global, 1+ = nested)
    int adr;           // Address/value/offset

    TabEntry() : identifier(""), link(0), obj(0), typ(0), ref(0), nrm(1), lev(0), adr(0) {}
};

struct BtabEntry {
    int last; // Index tab terakhir di block ini
    int lpar; // Index parameter terakhir (0 jika tidak ada parameter)
    int psze; // Parameter size (total ukuran parameter)
    int vsze; // Variable size (total ukuran variabel lokal)

    BtabEntry() : last(0), lpar(0), psze(0), vsze(0) {}
    BtabEntry(int l, int lp, int ps, int vs) : last(l), lpar(lp), psze(ps), vsze(vs) {}
};

struct AtabEntry {
    int xtyp; // Index type (type untuk indeks array)
    int etyp; // Element type (type elemen array)
    int eref; // Element reference (untuk nested array/record)
    int low;  // Lower bound
    int high; // Upper bound
    int elsz; // Element size
    int size; // Total array size

    AtabEntry() : xtyp(0), etyp(0), eref(0), low(0), high(0), elsz(0), size(0) {}
    AtabEntry(int x, int e, int er, int l, int h, int es, int s)
        : xtyp(x), etyp(e), eref(er), low(l), high(h), elsz(es), size(s) {}
};

class SymbolTable {
private:
    vector<TabEntry> tab;
    vector<BtabEntry> btab;
    vector<AtabEntry> atab;

    stack<int> display;
    int level;

    // Helper untuk quick lookup by name
    unordered_map<string, vector<int>> nameIndex;

    // Reserved words
    static const vector<string> RESERVED_WORDS;

public:
    SymbolTable();

    // ===== SCOPE MANAGEMENT =====

    int enterScope(int psize = 0, int vsize = 0);
    void exitScope();
    int getCurrentBlock() const;
    int getCurrentLevel() const;
};

```

```

// ===== SYMBOL OPERATIONS (TAB) =====

/**
 * Add new symbol to identifier table
 * @param entry Symbol entry to add
 * @return Index in tab where symbol was added
 */
int addSymbol(const TabEntry& entry);

/**
 * Lookup symbol by name
 * Searches from current scope upwards to global scope
 * @param name Identifier name
 * @param currentScopeOnly If true, only search in current block
 * @return Pointer to TabEntry if found, nullptr otherwise
 */
TabEntry* lookupSymbol(const string& name, bool currentScopeOnly = false);

/**
 * Get symbol by index
 * @param index Index in tab
 * @return Pointer to TabEntry
 */
TabEntry* getSymbol(int index);

/**
 * Get symbol by index (const version)
 */
const TabEntry* getSymbol(int index) const;

/**
 * Update symbol at given index
 * @param index Index in tab
 * @param entry New entry data
 */
void updateSymbol(int index, const TabEntry& entry);

/**
 * Get tab size
 */
int getTabSize() const;

// ===== BLOCK OPERATIONS (BTAB) =====

/**
 * Add new block
 * @param entry Block entry
 * @return Index in btab
 */
int addBlock(const BtabEntry& entry);

/**
 * Get block by index
 * @param index Index in btab
 * @return Pointer to BtabEntry
 */
BtabEntry* getBlock(int index);

/**
 * Get block by index (const version)
 */
const BtabEntry* getBlock(int index) const;

/**
 * Update block at given index
 * @param index Index in btab
 * @param entry New entry data
 */

```

```

void updateBlock(int index, const BtabEntry& entry);

/**
 * Get btab size
 */
int getBtabSize() const;

// ===== ARRAY OPERATIONS (ATAB) =====

/**
 * Add new array type
 * @param entry Array entry
 * @return Index in atab
 */
int addArray(const AtabEntry& entry);

/**
 * Get array by index
 * @param index Index in atab
 * @return Pointer to AtabEntry
 */
AtabEntry* getArray(int index);

/**
 * Get array by index (const version)
 */
const AtabEntry* getArray(int index) const;

/**
 * Get atab size
 */
int getAtabSize() const;

// ===== INITIALIZATION =====

/**
 * Initialize reserved words
 */
void initializeReservedWords();

/**
 * Initialize standard types and functions
 */
void initializeStandardIdentifiers();

// ===== DEBUGGING & PRINTING =====

/**
 * Print entire symbol table (tab)
 */
void printTab() const;

/**
 * Print block table (btab)
 */
void printBtab() const;

/**
 * Print array table (atab)
 */
void printAtab() const;

/**
 * Print all tables
 */
void printAll() const;

/**

```



```

    * Print symbols in specific block
    * @param blockIndex Index of block in btab
    */
    void printBlock(int blockIndex) const;

    /**
     * Print display stack (for debugging)
     */
    void printDisplay() const;
};

// Helper functions untuk convert enum ke string
string objectClassToString(int obj);
string typeCodeToString(int typ);

#endif

```

4. Kelas SemanticAnalyzer

SemanticAnalyzer mewarisi dari ASTVisitor untuk mengimplementasikan pola pengunjung, alih-alih menanamkan logika semantik di dalam setiap node AST, penganalisis menyediakan keluarga metode visitX(node) (satu per jenis node) dan node AST memaparkan panggilan accept(visitor) seragam yang mengirimkan ke metode kunjungan yang sesuai. SemanticAnalyzer melakukan traversal AST (visitor pattern) untuk mendekorasi node dengan informasi semantik dan memverifikasi aturan-aturan statis bahasa, yaitu membangun/menjaga symbol table per scope, menolak redeclaration, mendeteksi identifier tak terdefinisi, memeriksa kecocokan tipe pada assignment dan operator, memvalidasi parameter prosedur/fungsi, serta aturan khusus (misalnya loop variable). Analyzer menambahkan symbolTableIndex dan scopeLevel pada declaration node, menyimpulkan dan meneruskan tipe pada ekspresi, dan mengumpulkan error semantik dengan nomor baris.

```

#ifndef SEMANTIC_ANALYZER_H
#define SEMANTIC_ANALYZER_H

#include "ast_nodes.h"
#include "symbol_table.h"
#include <vector>
#include <string>
#include <memory>

class SemanticAnalyzer : public ASTVisitor {
private:
    SymbolTable& symbolTable;
    std::vector<std::string> errors;
    int currentLevel;
    int currentBlockIndex;

    void addError(const std::string& msg) {
        errors.push_back(msg);
    }

    void reportError(const std::string& message, int line = -1) {
        std::string fullMessage = "Semantic Error";
    }
};

```

```

        if (line > 0) {
            fullMessage += " at line " + std::to_string(line);
        }
        fullMessage += ": " + message;
        addError(fullMessage);
    }

public:
    SemanticAnalyzer(SymbolTable& symTable);

    void analyze(std::shared_ptr<ASTNode> root);

    const std::vector<std::string>& getErrors() const {
        return errors;
    }

    // Visitor methods
    void visitProgram(ProgramNode* node) override;
    void visitVarDecl(VarDeclNode* node) override;
    void visitConstDecl(ConstDeclNode* node) override;
    void visitTypeDecl(TypeDeclNode* node) override;
    void visitArrayType(ArrayTypeNode* node) override;
    void visitRecordType(RecordTypeNode* node) override;
    void visitProcedureDecl(ProcedureDeclNode* node) override;
    void visitFunctionDecl(FunctionDeclNode* node) override;
    void visitCompoundStatement(CompoundStatementNode* node) override;
    void visitAssign(AssignNode* node) override;
    void visitIf(IfNode* node) override;
    void visitWhile(WhileNode* node) override;
    void visitFor(ForNode* node) override;
    void visitProcCall(ProcCallNode* node) override;
    void visitBinOp(BinOpNode* node) override;
    void visitUnaryOp(UnaryOpNode* node) override;
    void visitNumber(NumberNode* node) override;
    void visitString(StringNode* node) override;
    void visitChar(CharNode* node) override;
    void visitBool(BoolNode* node) override;
    void visitVar(VarNode* node) override;
    void visitArrayAccess(ArrayAccessNode* node) override;
    void visitRecordAccess(RecordAccessNode* node) override;

private:
    // Helper methods for address calculation
    void calculateParameterAddresses(int blockIdx, size_t paramCount);
    void calculateVariableAddresses(int blockIdx);
};

#endif

```

d. Alur Kerja Program

Program semantic analysis dimulai di main. Ada dua tahap dalam semantic analysis, yaitu konstruksi AST (Abstract Syntax Tree) dan Type/Scope Checking.

1. AST Building

Masukan untuk ASTBuilder adalah parse tree (ParseNode) yang dihasilkan oleh parser. Method buildAST pada buildAST akan dipanggil, yang selanjutnya akan memanggil convertProgram. Secara rekursif, ASTBuilder akan secara rekursif method-method konversi ini. Tiap method konversi yang dipanggil memiliki alurnya masing-masing, yang disesuaikan dengan production rule dan semantic action yang terkait. Pada sebuah method konversi, children node yang sesuai akan dicari (sebagai contoh, pada convertProgram

akan dicari node program-header dan declaration-part) dan akan dipanggil metode konversi yang bersesuaian.

Secara umum, apa yang dilakukan di tahap ini adalah:

- Menyederhanakan struktur parse tree menjadi AST yang lebih semantik-oriented (ProgramNode, VarDeclNode, AssignNode, BinOpNode, NumberNode, dll.).
- Mengonversi konstruksi grammar menjadi node AST sesuai produksi: deklarasi, tipe, parameter, statement, ekspresi, akses array/record, dll.
- Mengekstrak literal (angka/string/char/boolean) dan membuat node literal yang tepat; untuk angka gunakan parsing yang aman (std::stoll/std::stod dalam try/catch).
- Mendeteksi operator (token RELATIONAL/ARITHMETIC/LOGICAL) dan membangun BinOp/UnaryOp dengan operand kiri/kanan.
- Menetapkan nomor baris ke setiap AST node (setLineNumber) dari ParseNode terkait agar error semantik dapat dilaporkan dengan lokasi.
- Melaporkan error konstruksi AST (mis. konstanta numerik invalid) tanpa melempar exception yang menghentikan proses.

Output atau keluarannya adalah AST terstruktur dan daftar error AST jika ada. Kemudian, keluaran tersebut diteruskan ke Type/Scope Checking.

2. Type/Scope Checking

SemanticAnalyzer menerima AST yang telah dibangun oleh ASTBuilder. Pertama, analyzer menginisialisasi SymbolTable (Tab, Btab, Atab), memasukkan reserved identifiers dan tipe bawaan, lalu memulai traversal dari root ProgramNode menggunakan pola visitor.

Pada visitProgram, analyzer membuat entri program di symbol table jika perlu, lalu memasuki scope global (enterScope). Selanjutnya ia memproses daftar deklarasi top-level satu per satu. Untuk tiap VarDecl/ConstDecl/TypeDecl: analyzer memeriksa redeclaration di scope saat ini (lookup currentScopeOnly), menambahkan TabEntry bila valid (addSymbol), dan menandai node deklarasi dengan symbolTableIndex dan scopeLevel. Untuk subprogram (Procedure/Function) ia membuat entri subprogram di parent scope, lalu memasuki scope baru untuk badan subprogram (enterScope), mendaftarkan parameter (dengan flag parameter/var-parameter), memproses deklarasi lokal, menghitung alamat/offset parameter dan variabel lokal, memeriksa tubuh subprogram, lalu keluar scope (exitScope).

Saat menelusuri statement dan ekspresi, analyzer melakukan type checking dan inferensi. Method visitAssign memastikan target dan ekspresi kanan kompatibel, visitBinOp/visitUnary memeriksa tipe operand sesuai operator (jika aritmetika maka numerik, jika logika maka boolean, dan jika relasional maka comparable) lalu menetapkan tipe hasil. visitVar mencari identifier di symbol table; bila tidak ditemukan, reportError(node->getLineNumber(), "... not declared"). Semua panggilan pelaporan error menggunakan nomor baris dari node AST sehingga pesan selalu terlokalisasi.

Untuk pemanggilan prosedur/fungsi, analyzer memeriksa keberadaan entry, jumlah dan

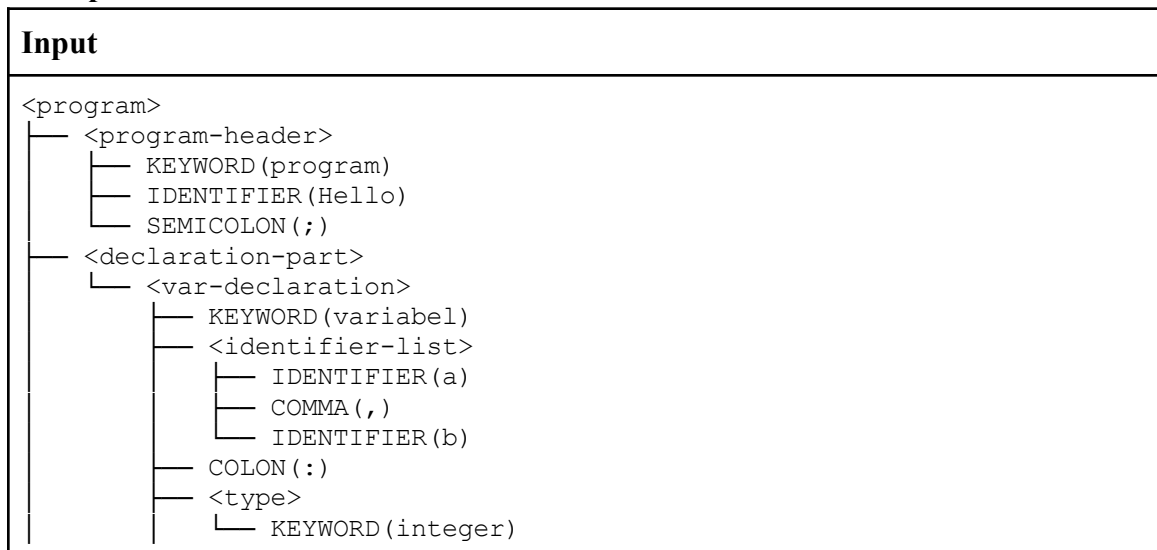
tipe argumen sesuai parameter, dan menetapkan tipe hasil pada fungsi. Untuk deklarasi tipe kompleks (array/record/subrange) ia menyimpan metadata di ATAB dan mencatat tipe pada TabEntry. Setelah traversal selesai, analyzer menghasilkan symbol tables yang terisi (TAB/BTAB/ATAB), decorated AST (node berisi tipe, symbolTableIndex, scopeLevel, lineNumber) dan daftar error.

Secara umum, apa yang dilakukan di SemanticAnalyzer yaitu:

- Mengelola SymbolTable (tab / btab / atab): enter/exit scope, addSymbol(), lookupSymbol().
- Mendaftarkan deklarasi (variabel/konstanta/tipe/prosedur/fungsi) ke symbol table dan menghias node deklarasi dengan symbolTableIndex dan scopeLevel.
- Mengunjungi semua node AST (visitor pattern) untuk:
 - Inference/propagasi tipe pada ekspresi dan literal.
 - Validasi aturan semantik: undeclared identifiers, redeclaration (current scope), type mismatch pada assignment/operasi, arity/type pada pemanggilan prosedur/fungsi, legalitas loop variable, dsb.
 - Menghitung alamat/offset untuk parameter dan variabel lokal (calculateParameterAddresses / calculateVariableAddresses) dan menyimpan di TabEntry.
 - Menangani operator: memastikan operand kompatibel (mis. aritmetika hanya numeric; logika hanya boolean; relasional ke tipe yang comparable) dan menentukan tipe hasil (integer/real/boolean).
 - Mengumpulkan error semantik terstruktur yang menyertakan nomor baris dari node (reportError dengan line number).
 - Membatasi cascading errors: jika sub-ekspresi sudah UNKNOWN, jangan hasilkan banyak error turunan.

3. Pengujian

a. dasar.pas



```

└─ SEMICOLON(;)
└─ <compound-statement>
└─ KEYWORD(mulai)
└─ <statement-list>
└─ <assignment-statement>
└─ IDENTIFIER(a)
└─ ASSIGN_OPERATOR(=)
└─ <expression>
└─ <simple-expression>
└─ <term>
└─ <factor>
└─ NUMBER(5)
└─ SEMICOLON(;)
└─ <assignment-statement>
└─ IDENTIFIER(b)
└─ ASSIGN_OPERATOR(=)
└─ <expression>
└─ <simple-expression>
└─ <term>
└─ <factor>
└─ IDENTIFIER(a)
└─ ARITHMETIC_OPERATOR(+)
└─ <term>
└─ <factor>
└─ NUMBER(10)
└─ SEMICOLON(;)
└─ <procedure-call>
└─ IDENTIFIER(writeln)
└─ LPARENTHESIS(( )
└─ <parameter-list>
└─ <expression>
└─ <simple-expression>
└─ <term>
└─ <factor>
└─ STRING_LITERAL(Result = )
└─ COMMA(,)
└─ <expression>
└─ <simple-expression>
└─ <term>
└─ <factor>
└─ IDENTIFIER(b)
└─ RPARENTHESIS())
└─ SEMICOLON(;)
└─ KEYWORD(selesai)
└─ DOT(.)

```

Output

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr
0	dan	0	RESERVED	NOTYP	0	1	0	0
1	larik	0	RESERVED	NOTYP	0	1	0	0
2	mulai	1	RESERVED	NOTYP	0	1	0	0

3	case	2	RESERVED	NOTYP	0	1	0	0
4	konstanta	3	RESERVED	NOTYP	0	1	0	0
5	bagi	4	RESERVED	NOTYP	0	1	0	0
6	turun_ke	5	RESERVED	NOTYP	0	1	0	0
7	lakukan	6	RESERVED	NOTYP	0	1	0	0
8	selain_itu	7	RESERVED	NOTYP	0	1	0	0
9	selesai	8	RESERVED	NOTYP	0	1	0	0
10	untuk	9	RESERVED	NOTYP	0	1	0	0
11	fungsi	10	RESERVED	NOTYP	0	1	0	0
12	jika	11	RESERVED	NOTYP	0	1	0	0
13	mod	12	RESERVED	NOTYP	0	1	0	0
14	tidak	13	RESERVED	NOTYP	0	1	0	0
15	dari	14	RESERVED	NOTYP	0	1	0	0
16	atau	15	RESERVED	NOTYP	0	1	0	0
17	prosedur	16	RESERVED	NOTYP	0	1	0	0
18	program	17	RESERVED	NOTYP	0	1	0	0
19	record	18	RESERVED	NOTYP	0	1	0	0
20	ulangi	19	RESERVED	NOTYP	0	1	0	0
21	string	20	RESERVED	NOTYP	0	1	0	0
22	maka	21	RESERVED	NOTYP	0	1	0	0
23	ke	22	RESERVED	NOTYP	0	1	0	0
24	tipe	23	RESERVED	NOTYP	0	1	0	0
25	sampai	24	RESERVED	NOTYP	0	1	0	0
26	variabel	25	RESERVED	NOTYP	0	1	0	0
27	selama	26	RESERVED	NOTYP	0	1	0	0
28	packed	27	RESERVED	NOTYP	0	1	0	0
29	integer	-1	TYPE	INT	0	1	0	1
30	real	29	TYPE	REAL	0	1	0	1
31	boolean	30	TYPE	BOOL	0	1	0	1
32	char	31	TYPE	CHAR	0	1	0	1
33	true	32	CONST	BOOL	1	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	read	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	1
37	write	36	PROC	NOTYP	0	1	0	2
38	writeln	37	PROC	NOTYP	0	1	0	3
39	Hello	38	PROGRAM	NOTYP	0	1	0	0
40	a	39	VAR	INT	0	1	0	0
41	b	40	VAR	INT	0	1	0	1

=== BTAB (Block Table) ===

Idx	Last	Lpar	Psze	Vsze
0	41	0	0	2
1	41	0	0	0

DECORATED AST:

ProgramNode(name: 'Hello', tab_index: 39)

Declarations:

VarDecl(name: 'a', type: integer, tab_index: 40, level: 0)

VarDecl(name: 'b', type: integer, tab_index: 41, level: 0)

Body:

CompoundStatement

Assign

Target:

```

    Var(name: 'a', tab_index: 40, type: integer)
Value:
    Number(int: 5, type: integer)
Assign
Target:
    Var(name: 'b', tab_index: 41, type: integer)
Value:
    BinOp(op: '+', type: integer)
    Left:
        Var(name: 'a', tab_index: 40, type: integer)
    Right:
        Number(int: 10, type: integer)
ProcCall(name: 'writeln', tab_index: 38)
Arguments:
    String(value: "Result = ")
    Var(name: 'b', tab_index: 41, type: integer)

```

Bukti

```

Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-3/dasar.pas
-----
Starting syntax analysis...
-----
Parse Tree:
-----
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER>Hello)
│   └── SEMICOLON(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── KEYWORD(variabel)
│       ├── <identifier-list>
│       │   ├── IDENTIFIER(a)
│       │   ├── COMMA(,)
│       │   └── IDENTIFIER(b)
│       ├── COLON(:)
│       ├── <type>
│       │   └── KEYWORD(integer)
│       └── SEMICOLON(; )

```

```

-----
DECORATED AST:
-----
ProgramNode(name: 'Hello', tab_index: 39)
  Declarations:
    VarDecl(name: 'a', type: integer, tab_index: 40, level: 0)
    VarDecl(name: 'b', type: integer, tab_index: 41, level: 0)
  Body:
    CompoundStatement
      Assign
        Target:
          Var(name: 'a', tab_index: 40, type: integer)
        Value:
          Number(int: 5, type: integer)
      Assign
        Target:
          Var(name: 'b', tab_index: 41, type: integer)
        Value:
          BinOp(op: '+', type: integer)
            Left:
              Var(name: 'a', tab_index: 40, type: integer)
            Right:
              Number(int: 10, type: integer)
      ProcCall(name: 'writeln', tab_index: 38)
        Arguments:
          String(value: "Result = ")
          Var(name: 'b', tab_index: 41, type: integer)
-----
Semantic analysis completed!
-----

```

b. test_var_param.pas

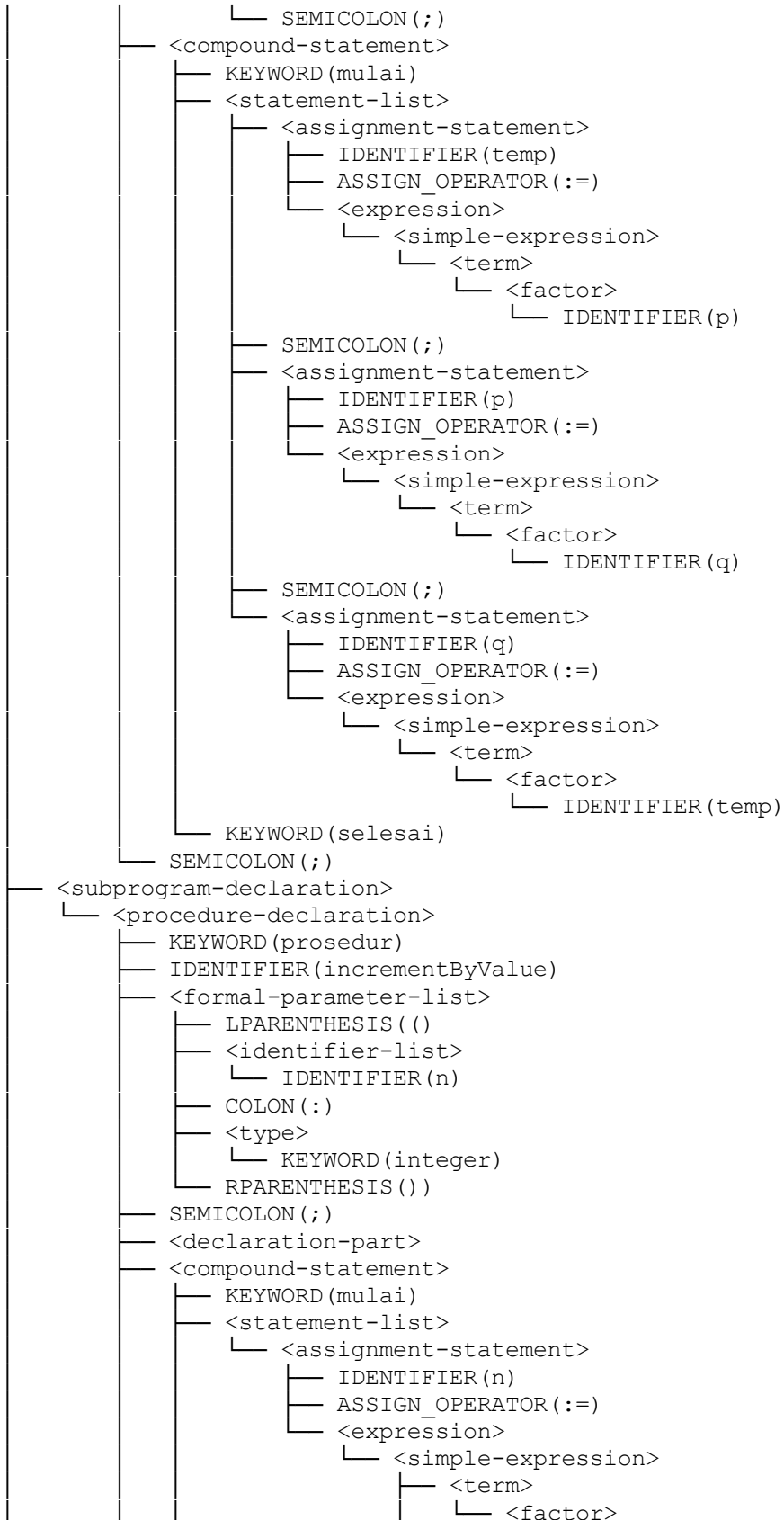
Input

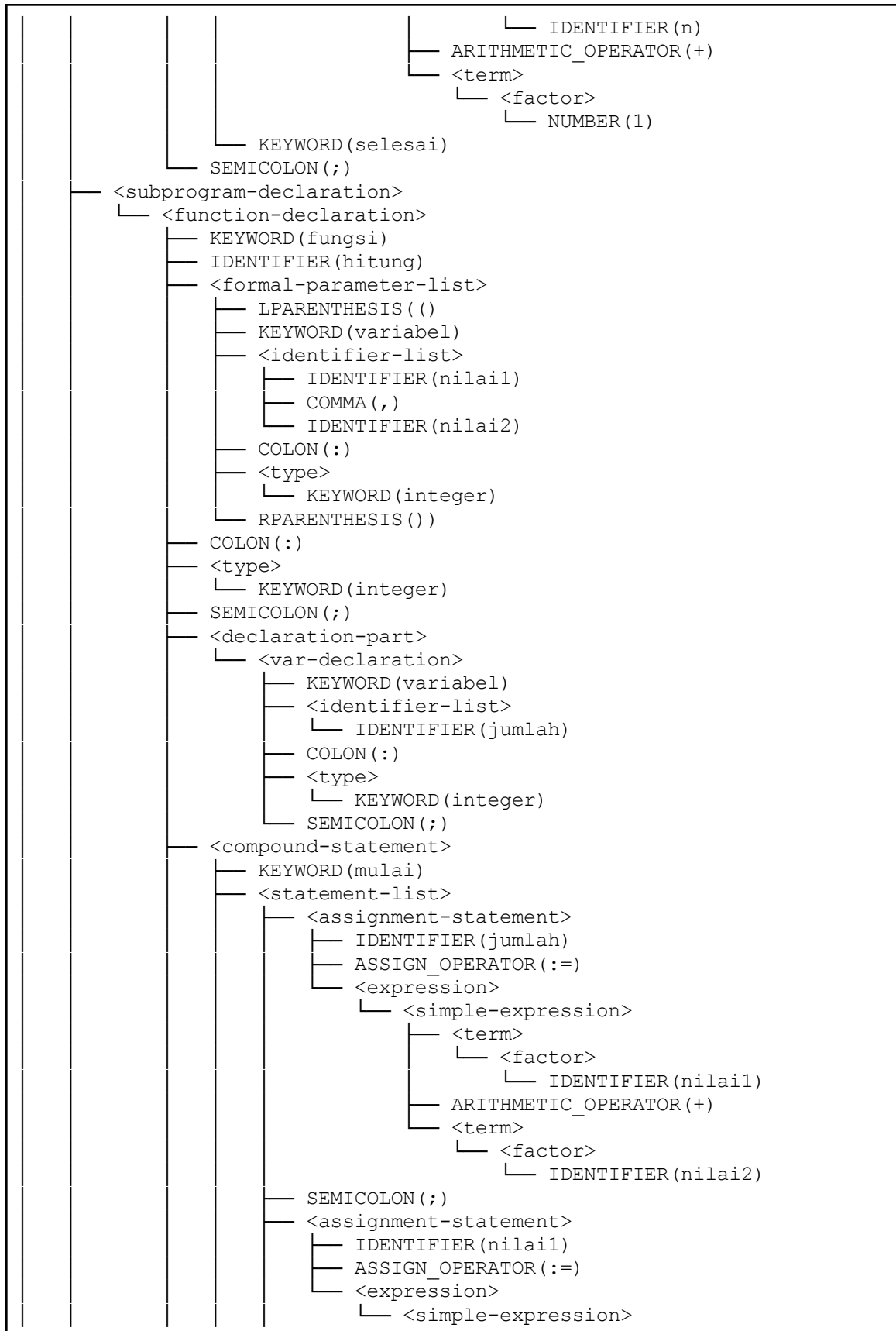
```

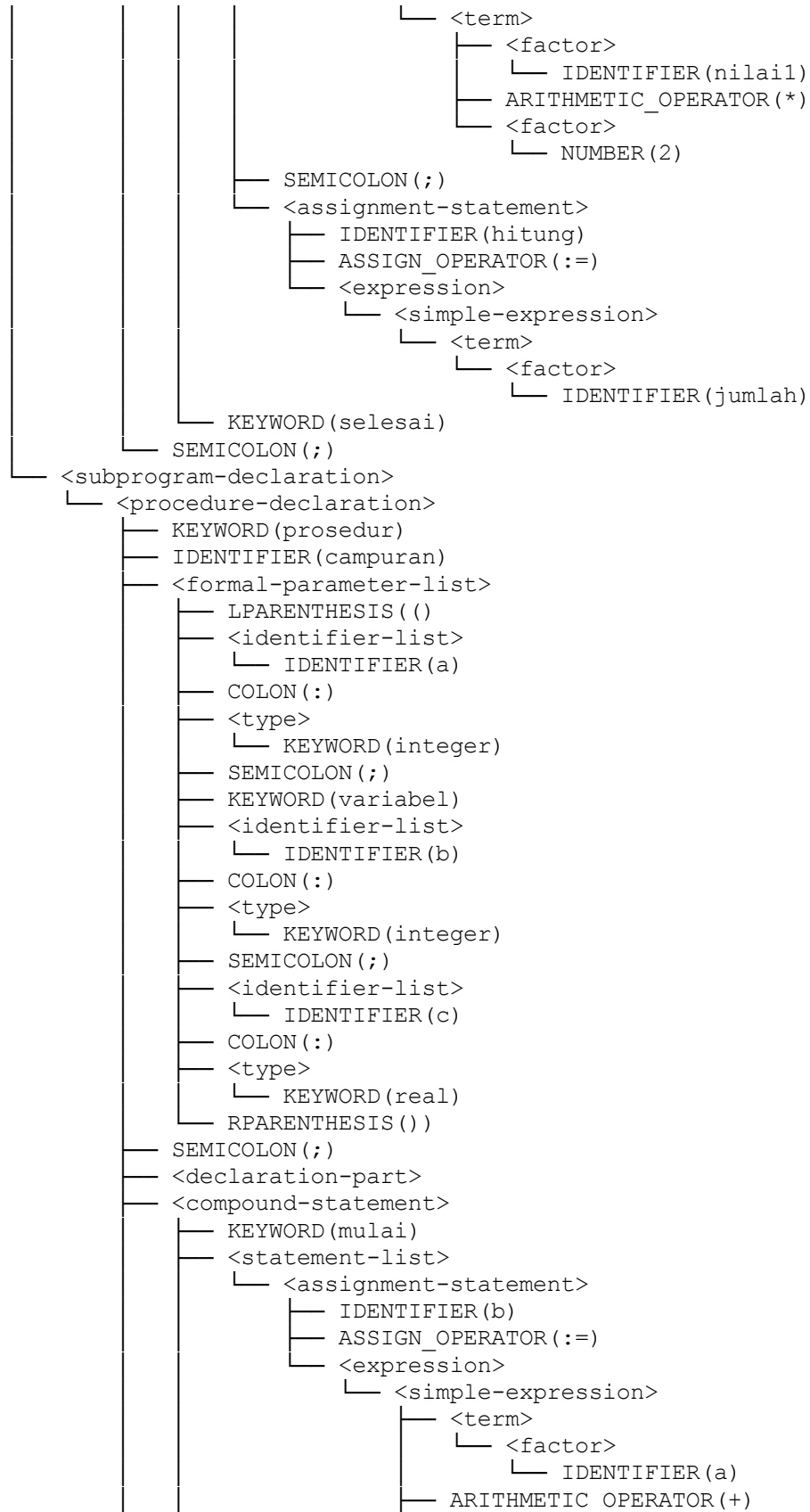
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestVarParameter)
│   └── SEMICOLON(;)
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── KEYWORD(konstanta)
│   │   ├── IDENTIFIER(MAX_SIZE)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   ├── <constant-value>(100)
│   │   └── SEMICOLON(;)

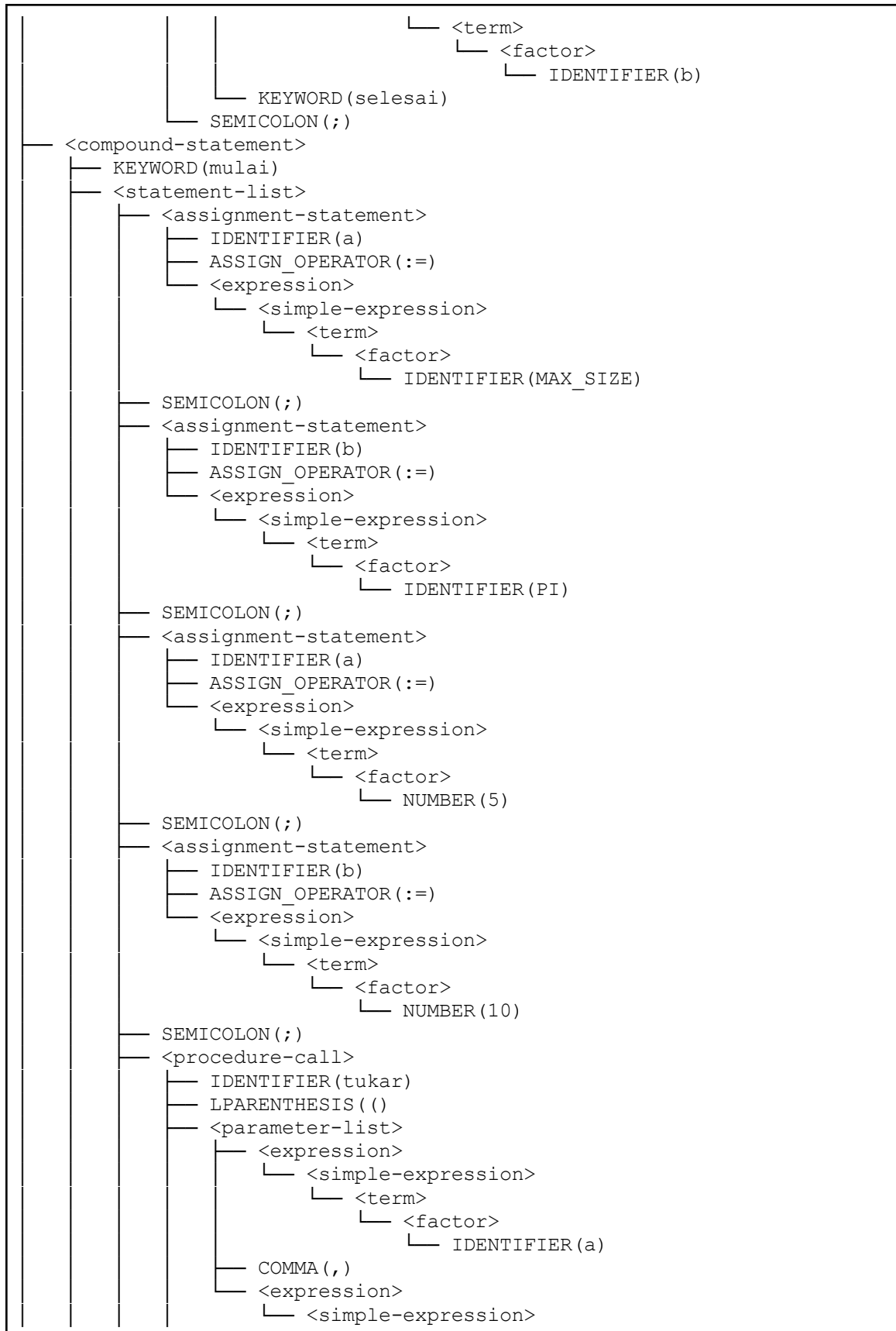
```


- IDENTIFIER (PI)
- RELATIONAL_OPERATOR (=)
- <constant-value> (314)
- SEMICOLON (;)
- IDENTIFIER (adalah_benar)
- RELATIONAL_OPERATOR (=)
- <constant-value> (true)
- SEMICOLON (;)
- <var-declaration>
 - KEYWORD (variabel)
 - <identifier-list>
 - IDENTIFIER (a)
 - COMMA (,)
 - IDENTIFIER (b)
 - COLON (:)
 - <type>
 - KEYWORD (integer)
 - SEMICOLON (;)
 - <identifier-list>
 - IDENTIFIER (x)
 - COMMA (,)
 - IDENTIFIER (y)
 - COLON (:)
 - <type>
 - KEYWORD (real)
 - SEMICOLON (;)
 - <identifier-list>
 - IDENTIFIER (hasil)
 - COLON (:)
 - <type>
 - KEYWORD (integer)
 - SEMICOLON (;)
- <subprogram-declaration>
 - <procedure-declaration>
 - KEYWORD (prosedur)
 - IDENTIFIER (tukar)
 - <formal-parameter-list>
 - LPARENTHESIS (()
 - KEYWORD (variabel)
 - <identifier-list>
 - IDENTIFIER (p)
 - COMMA (,)
 - IDENTIFIER (q)
 - COLON (:)
 - <type>
 - KEYWORD (integer)
 - RPARENTHESIS ())
 - SEMICOLON (;)
 - <declaration-part>
 - <var-declaration>
 - KEYWORD (variabel)
 - <identifier-list>
 - IDENTIFIER (temp)
 - COLON (:)
 - <type>
 - KEYWORD (integer)





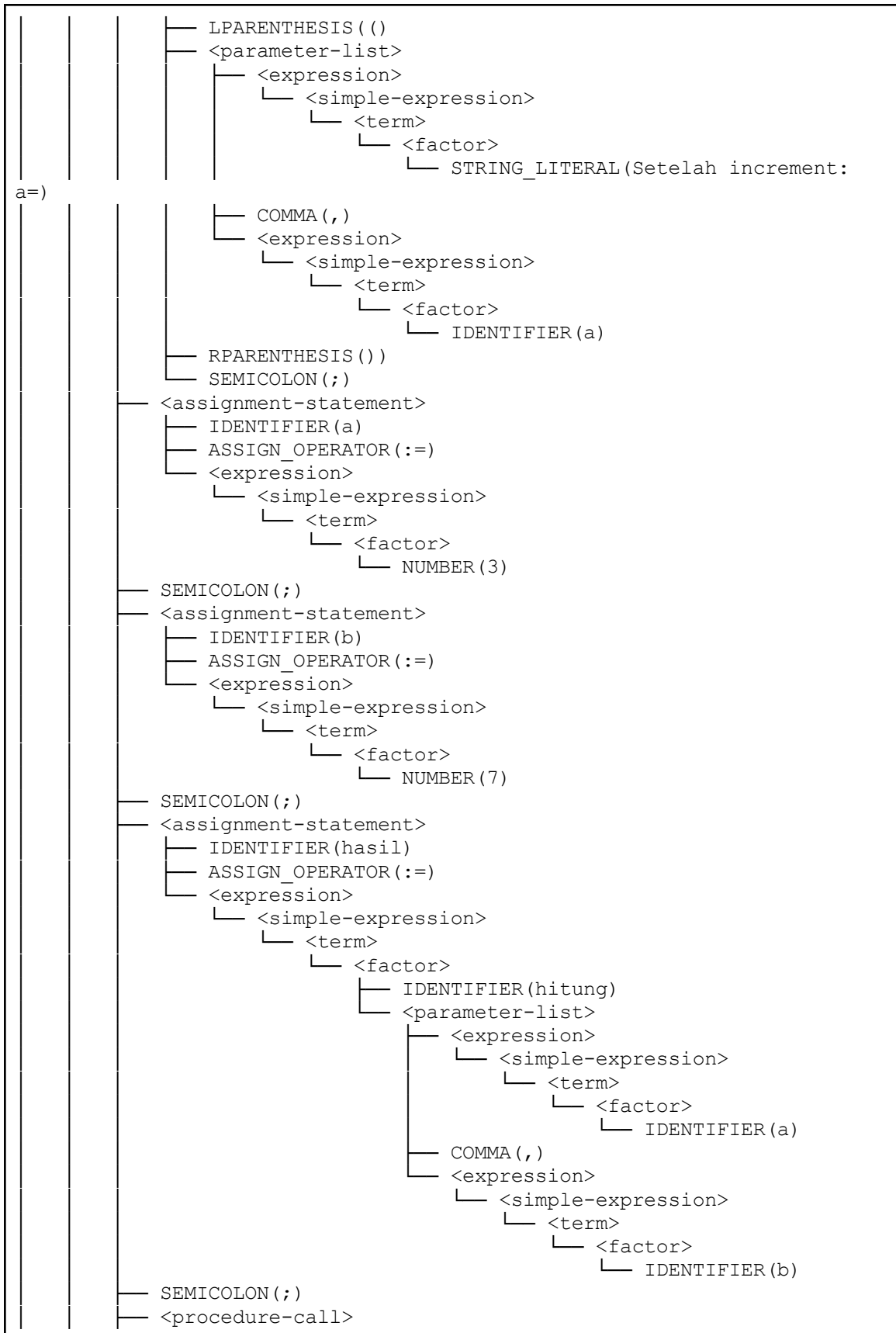




```

      └─ <term>
        └─ <factor>
          └─ IDENTIFIER(b)
    └─ RPARENTHESIS())
    └─ SEMICOLON(;)
  └─ <procedure-call>
    └─ IDENTIFIER(writeln)
    └─ LPARENTHESIS(())
    └─ <parameter-list>
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ STRING_LITERAL(Setelah tukar: a=)
      └─ COMMA(,)
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ IDENTIFIER(a)
      └─ COMMA(,)
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ STRING_LITERAL(, b=)
      └─ COMMA(,)
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ IDENTIFIER(b)
    └─ RPARENTHESIS())
    └─ SEMICOLON(;)
  └─ <assignment-statement>
    └─ IDENTIFIER(a)
    └─ ASSIGN_OPERATOR(:=)
    └─ <expression>
      └─ <simple-expression>
        └─ <term>
          └─ <factor>
            └─ NUMBER(5)
    └─ SEMICOLON(;)
  └─ <procedure-call>
    └─ IDENTIFIER(incrementByValue)
    └─ LPARENTHESIS(())
    └─ <parameter-list>
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ IDENTIFIER(a)
    └─ RPARENTHESIS())
    └─ SEMICOLON(;)
  └─ <procedure-call>
    └─ IDENTIFIER(writeln)

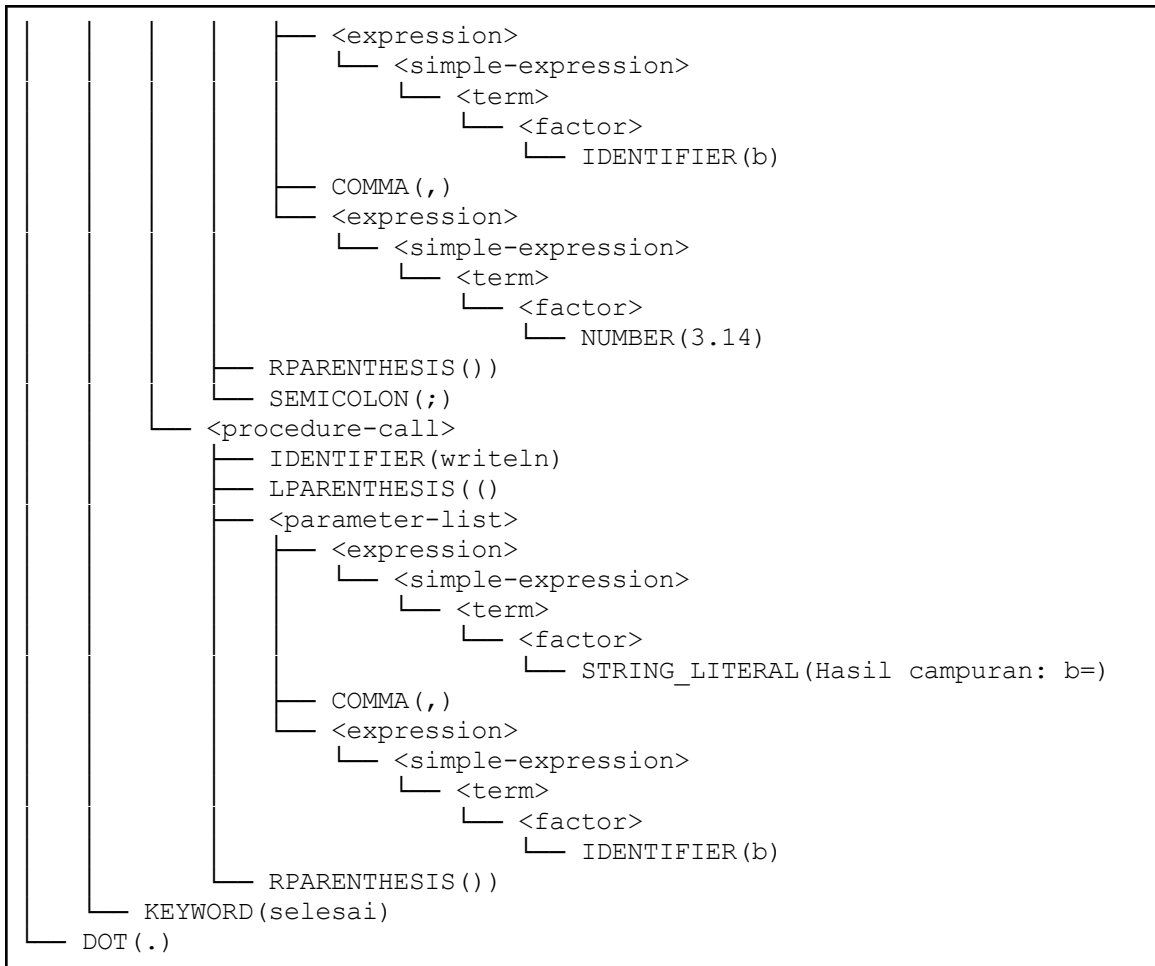
```



```

IDENTIFIER(writeln)
LPARENTHESIS ( ()
<parameter-list>
  <expression>
    <simple-expression>
      <term>
        <factor>
          STRING_LITERAL (Hasil=)
  COMMA (,)
  <expression>
    <simple-expression>
      <term>
        <factor>
          IDENTIFIER (hasil)
  COMMA (,)
  <expression>
    <simple-expression>
      <term>
        <factor>
          STRING_LITERAL (, a=)
  COMMA (,)
  <expression>
    <simple-expression>
      <term>
        <factor>
          IDENTIFIER (a)
RPARENTHESIS ())
SEMICOLON (;)
<assignment-statement>
  IDENTIFIER (a)
  ASSIGN_OPERATOR (:=)
  <expression>
    <simple-expression>
      <term>
        <factor>
          NUMBER (10)
SEMICOLON (;)
<assignment-statement>
  IDENTIFIER (b)
  ASSIGN_OPERATOR (:=)
  <expression>
    <simple-expression>
      <term>
        <factor>
          NUMBER (20)
SEMICOLON (;)
<procedure-call>
  IDENTIFIER (campuran)
  LPARENTHESIS ( ()
  <parameter-list>
    <expression>
      <simple-expression>
        <term>
          <factor>
            IDENTIFIER (a)
  COMMA (,)

```

Output

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr

0	dan	0	RESERVED	NOTYP	0	1	0	0
1	larik	0	RESERVED	NOTYP	0	1	0	0
2	mulai	1	RESERVED	NOTYP	0	1	0	0
3	case	2	RESERVED	NOTYP	0	1	0	0
4	konstanta	3	RESERVED	NOTYP	0	1	0	0
5	bagi	4	RESERVED	NOTYP	0	1	0	0
6	turun_ke	5	RESERVED	NOTYP	0	1	0	0
7	lakukan	6	RESERVED	NOTYP	0	1	0	0
8	selain_itu	7	RESERVED	NOTYP	0	1	0	0
9	selesai	8	RESERVED	NOTYP	0	1	0	0
10	untuk	9	RESERVED	NOTYP	0	1	0	0
11	fungsi	10	RESERVED	NOTYP	0	1	0	0
12	jika	11	RESERVED	NOTYP	0	1	0	0
13	mod	12	RESERVED	NOTYP	0	1	0	0
14	tidak	13	RESERVED	NOTYP	0	1	0	0
15	dari	14	RESERVED	NOTYP	0	1	0	0
16	atau	15	RESERVED	NOTYP	0	1	0	0

17	prosedur	16	RESERVED	NOTYP	0	1	0	0
18	program	17	RESERVED	NOTYP	0	1	0	0
19	record	18	RESERVED	NOTYP	0	1	0	0
20	ulangi	19	RESERVED	NOTYP	0	1	0	0
21	string	20	RESERVED	NOTYP	0	1	0	0
22	maka	21	RESERVED	NOTYP	0	1	0	0
23	ke	22	RESERVED	NOTYP	0	1	0	0
24	tipe	23	RESERVED	NOTYP	0	1	0	0
25	sampai	24	RESERVED	NOTYP	0	1	0	0
26	variabel	25	RESERVED	NOTYP	0	1	0	0
27	selama	26	RESERVED	NOTYP	0	1	0	0
28	packed	27	RESERVED	NOTYP	0	1	0	0
29	integer	-1	TYPE	INT	0	1	0	1
30	real	29	TYPE	REAL	0	1	0	1
31	boolean	30	TYPE	BOOL	0	1	0	1
32	char	31	TYPE	CHAR	0	1	0	1
33	true	32	CONST	BOOL	1	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	read	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	1
37	write	36	PROC	NOTYP	0	1	0	2
38	writeln	37	PROC	NOTYP	0	1	0	3
39	TestVarParameter	38	PROGRAM	NOTYP	0	1	0	0
40	MAX_SIZE	39	CONST	INT	0	1	0	100
41	PI	40	CONST	INT	0	1	0	314
42	adalah_benar	41	CONST	BOOL	0	1	0	1
43	a	42	VAR	INT	0	1	0	0
44	b	43	VAR	INT	0	1	0	1
45	x	44	VAR	REAL	0	1	0	2
46	y	45	VAR	REAL	0	1	0	3
47	hasil	46	VAR	INT	0	1	0	4
48	tukar	47	PROC	NOTYP	1	1	0	1
49	p	48	VAR	INT	0	0	1	0
50	q	49	VAR	INT	0	0	1	1
51	temp	50	VAR	INT	0	1	1	0
52	incrementByValue	51	PROC	NOTYP	2	1	0	2
53	n	52	VAR	INT	0	1	1	0
54	hitung	53	FUNC	INT	3	1	0	3
55	nilai1	54	VAR	INT	0	0	1	0
56	nilai2	55	VAR	INT	0	0	1	1
57	jumlah	56	VAR	INT	0	1	1	0
58	campuran	57	PROC	NOTYP	4	1	0	4
59	a	58	VAR	INT	0	1	1	0
60	b	59	VAR	INT	0	0	1	1
61	c	60	VAR	REAL	0	1	1	2

=== BTAB (Block Table) ===

Idx	Last	Lpar	Psze	Vsze
0	47	0	0	5
1	51	50	2	1
2	53	53	1	0
3	57	56	2	1
4	61	61	3	0
5	61	0	0	0

DECORATED AST:

```
-----
ProgramNode(name: 'TestVarParameter', tab_index: 39)
  Declarations:
    ConstDecl(name: 'MAX_SIZE', type: integer, tab_index: 40)
      Value:
        Number(int: 100, type: integer)
    ConstDecl(name: 'PI', type: integer, tab_index: 41)
      Value:
        Number(int: 314, type: integer)
    ConstDecl(name: 'adalah_benar', type: boolean, tab_index: 42)
      Value:
        Bool(value: true)
    VarDecl(name: 'a', type: integer, tab_index: 43, level: 0)
    VarDecl(name: 'b', type: integer, tab_index: 44, level: 0)
    VarDecl(name: 'x', type: real, tab_index: 45, level: 0)
    VarDecl(name: 'y', type: real, tab_index: 46, level: 0)
    VarDecl(name: 'hasil', type: integer, tab_index: 47, level: 0)
    ProcedureDecl(name: 'tukar', tab_index: 48)
      Parameters:
        VarDecl(name: 'p', type: integer, tab_index: 49, level: 1,
parameter, var)
        VarDecl(name: 'q', type: integer, tab_index: 50, level: 1,
parameter, var)
      Declarations:
        VarDecl(name: 'temp', type: integer, tab_index: 51, level: 1)
      Body:
        CompoundStatement
          Assign
            Target:
              Var(name: 'temp', tab_index: 51, type: integer, level: 1)
            Value:
              Var(name: 'p', tab_index: 49, type: integer, level: 1)
          Assign
            Target:
              Var(name: 'p', tab_index: 49, type: integer, level: 1)
            Value:
              Var(name: 'q', tab_index: 50, type: integer, level: 1)
          Assign
            Target:
              Var(name: 'q', tab_index: 50, type: integer, level: 1)
            Value:
              Var(name: 'temp', tab_index: 51, type: integer, level: 1)
        ProcedureDecl(name: 'incrementByValue', tab_index: 52)
          Parameters:
            VarDecl(name: 'n', type: integer, tab_index: 53, level: 1,
parameter)
          Body:
            CompoundStatement
              Assign
                Target:
                  Var(name: 'n', tab_index: 53, type: integer, level: 1)
                Value:
                  BinOp(op: '+', type: integer)
                  Left:
                    Var(name: 'n', tab_index: 53, type: integer, level: 1)
```



```

        Left:
            Var(name: 'a', tab_index: 43, type: integer, level: 1)
        Right:
            Var(name: 'b', tab_index: 44, type: integer, level: 1)
Body:
CompoundStatement
Assign
    Target:
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
    Value:
        Var(name: 'MAX_SIZE', tab_index: 40, type: integer)
Assign
    Target:
        Var(name: 'b', tab_index: 44, type: integer, level: 1)
    Value:
        Var(name: 'PI', tab_index: 41, type: integer)
Assign
    Target:
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
    Value:
        Number(int: 5, type: integer)
Assign
    Target:
        Var(name: 'b', tab_index: 44, type: integer, level: 1)
    Value:
        Number(int: 10, type: integer)
ProcCall(name: 'tukar', tab_index: 48)
    Arguments:
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
        Var(name: 'b', tab_index: 44, type: integer, level: 1)
ProcCall(name: 'writeln', tab_index: 38)
    Arguments:
        String(value: "Setelah tukar: a=")
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
        String(value: ", b=")
        Var(name: 'b', tab_index: 44, type: integer, level: 1)
Assign
    Target:
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
    Value:
        Number(int: 5, type: integer)
ProcCall(name: 'incrementByValue', tab_index: 52)
    Arguments:
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
ProcCall(name: 'writeln', tab_index: 38)
    Arguments:
        String(value: "Setelah increment: a=")
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
Assign
    Target:
        Var(name: 'a', tab_index: 43, type: integer, level: 1)
    Value:
        Number(int: 3, type: integer)
Assign
    Target:
        Var(name: 'b', tab_index: 44, type: integer, level: 1)

```

```

    Value:
      Number(int: 7, type: integer)
Assign
  Target:
    Var(name: 'hasil', tab_index: 47, type: integer)
  Value:
    Var(name: 'hitung', tab_index: 54, type: integer)
ProcCall(name: 'writeln', tab_index: 38)
  Arguments:
    String(value: "Hasil=")
    Var(name: 'hasil', tab_index: 47, type: integer)
    String(value: ", a=")
    Var(name: 'a', tab_index: 43, type: integer, level: 1)
Assign
  Target:
    Var(name: 'a', tab_index: 43, type: integer, level: 1)
  Value:
    Number(int: 10, type: integer)
Assign
  Target:
    Var(name: 'b', tab_index: 44, type: integer, level: 1)
  Value:
    Number(int: 20, type: integer)
ProcCall(name: 'campuran', tab_index: 58)
  Arguments:
    Var(name: 'a', tab_index: 43, type: integer, level: 1)
    Var(name: 'b', tab_index: 44, type: integer, level: 1)
    Number(real: 3.14, type: real)
ProcCall(name: 'writeln', tab_index: 38)
  Arguments:
    String(value: "Hasil campuran: b=")
    Var(name: 'b', tab_index: 44, type: integer, level: 1)

```

Bukti

```
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-3/test_var_param.pas
```

```
-----
Starting syntax analysis...
-----
```

```
Parse Tree:
```

```
-----
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestVarParameter)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <const-declaration>
    │   ├── KEYWORD(konstanta)
    │   ├── IDENTIFIER(MAX_SIZE)
    │   ├── RELATIONAL_OPERATOR(=)
    │   ├── <constant-value>(100)
    │   ├── SEMICOLON(;)
    │   └── IDENTIFIER(PI)
    └──
```

```
Assign
```

```
Target:
```

```
Var(name: 'a', tab_index: 43, type: integer, level: 1)
```

```
Value:
```

```
Number(int: 10, type: integer)
```

```
Assign
```

```
Target:
```

```
Var(name: 'b', tab_index: 44, type: integer, level: 1)
```

```
Value:
```

```
Number(int: 20, type: integer)
```

```
ProcCall(name: 'campuran', tab_index: 58)
```

```
Arguments:
```

```
Var(name: 'a', tab_index: 43, type: integer, level: 1)
```

```
Var(name: 'b', tab_index: 44, type: integer, level: 1)
```

```
Number(real: 3.14, type: real)
```

```
ProcCall(name: 'writeln', tab_index: 38)
```

```
Arguments:
```

```
String(value: "Hasil campuran: b=")
```

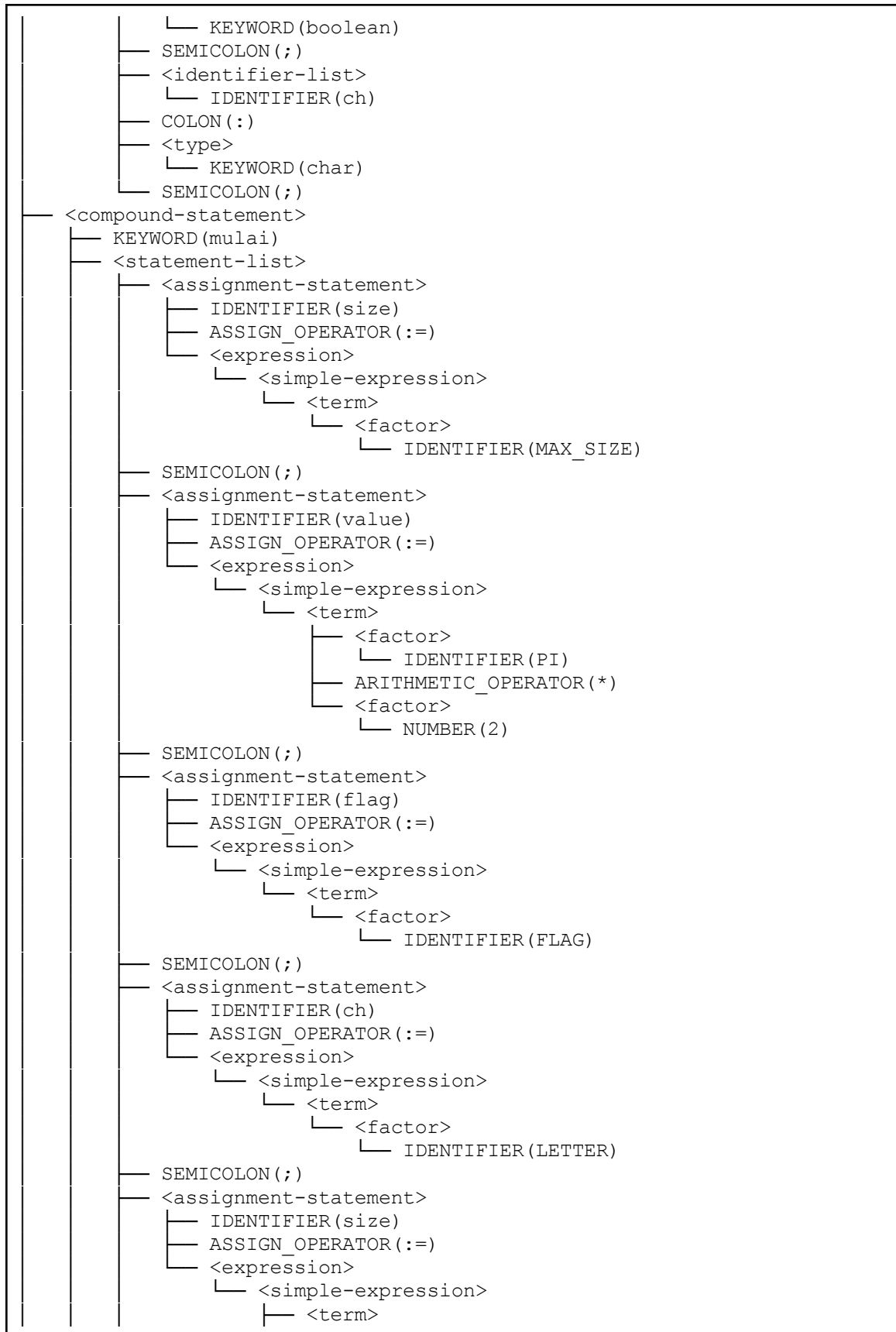
```
Var(name: 'b', tab_index: 44, type: integer, level: 1)
```

```
-----
Semantic analysis completed!
-----
```

c. test_constants.pas

Input

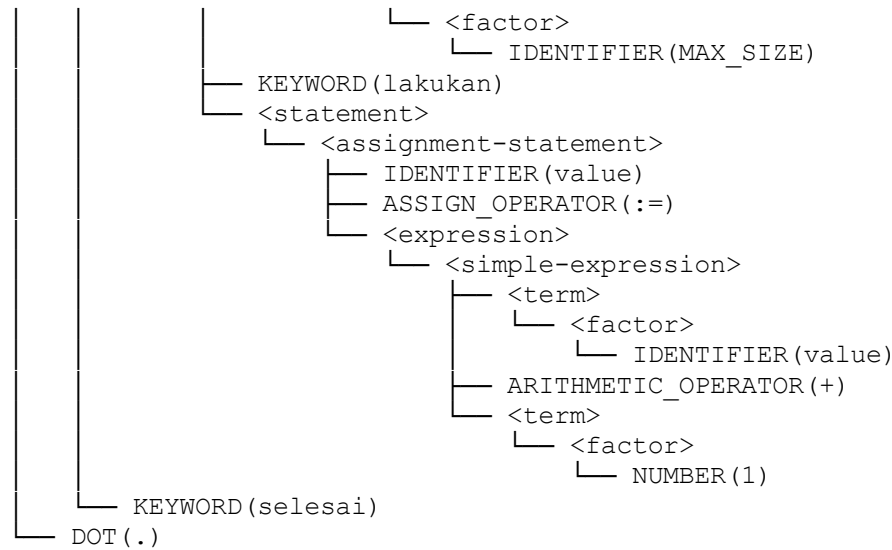
```
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestConstants)
│   └── SEMICOLON(;)
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── KEYWORD(konstanta)
│   │   ├── IDENTIFIER(MAX_SIZE)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   ├── <constant-value>(100)
│   │   ├── SEMICOLON(;)
│   │   ├── IDENTIFIER(MIN_VALUE)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   ├── <constant-value>(0)
│   │   ├── SEMICOLON(;)
│   │   ├── IDENTIFIER(PI)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   ├── <constant-value>(3.14)
│   │   ├── SEMICOLON(;)
│   │   ├── IDENTIFIER(GREETING)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   ├── <constant-value>(Hello)
│   │   ├── SEMICOLON(;)
│   │   ├── IDENTIFIER(FLAG)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   ├── <constant-value>(true)
│   │   ├── SEMICOLON(;)
│   │   ├── IDENTIFIER(LETTER)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   ├── <constant-value>(A)
│   │   └── SEMICOLON(;)
│   └── <var-declaration>
│       ├── KEYWORD(variabel)
│       ├── <identifier-list>
│       │   └── IDENTIFIER(size)
│       ├── COLON(:)
│       ├── <type>
│       │   └── KEYWORD(integer)
│       ├── SEMICOLON(;)
│       ├── <identifier-list>
│       │   └── IDENTIFIER(value)
│       ├── COLON(:)
│       ├── <type>
│       │   └── KEYWORD(real)
│       ├── SEMICOLON(;)
│       ├── <identifier-list>
│       │   └── IDENTIFIER(flag)
│       ├── COLON(:)
│       └── <type>
```

```

└── <factor>
    └── IDENTIFIER(MAX_SIZE)
└── ARITHMETIC_OPERATOR(-)
└── <term>
    └── <factor>
        └── IDENTIFIER(MIN_VALUE)
└── SEMICOLON(;)
└── <assignment-statement>
    ├── IDENTIFIER(value)
    ├── ASSIGN_OPERATOR(:=)
    └── <expression>
        └── <simple-expression>
            ├── <term>
            │   ├── <factor>
            │   │   └── IDENTIFIER(PI)
            │   └── ARITHMETIC_OPERATOR(+)
            └── <term>
                └── <factor>
                    └── NUMBER(1.5)
└── SEMICOLON(;)
└── <if-statement>
    ├── KEYWORD(jika)
    ├── <expression>
    │   ├── <simple-expression>
    │   │   ├── <term>
    │   │   │   └── <factor>
    │   │   │       └── IDENTIFIER(size)
    │   └── RELATIONAL_OPERATOR(>)
    │   └── <simple-expression>
    │       ├── <term>
    │       │   └── <factor>
    │       │       └── IDENTIFIER(MAX_SIZE)
    └── KEYWORD(maka)
        └── <then-statement>
            └── <assignment-statement>
                ├── IDENTIFIER(size)
                ├── ASSIGN_OPERATOR(:=)
                └── <expression>
                    └── <simple-expression>
                        └── <term>
                            └── <factor>
                                └── IDENTIFIER(MAX_SIZE)
└── SEMICOLON(;)
└── <for-statement>
    ├── KEYWORD(untuk)
    ├── IDENTIFIER(size)
    ├── ASSIGN_OPERATOR(:=)
    ├── <expression>
    │   └── <simple-expression>
    │       └── <term>
    │           └── <factor>
    │               └── IDENTIFIER(MIN_VALUE)
    ├── KEYWORD(ke)
    └── <expression>
        └── <simple-expression>
            └── <term>

```



Output

=== TAB (Identifier Table) ===

Idx	Identifier	Link	Obj	Type	Ref	Nrm	Lev	Adr

0	dan	0	RESERVED	NOTYP	0	1	0	0
1	larik	0	RESERVED	NOTYP	0	1	0	0
2	mulai	1	RESERVED	NOTYP	0	1	0	0
3	case	2	RESERVED	NOTYP	0	1	0	0
4	konstanta	3	RESERVED	NOTYP	0	1	0	0
5	bagi	4	RESERVED	NOTYP	0	1	0	0
6	turun_ke	5	RESERVED	NOTYP	0	1	0	0
7	lakukan	6	RESERVED	NOTYP	0	1	0	0
8	selain_itu	7	RESERVED	NOTYP	0	1	0	0
9	selesai	8	RESERVED	NOTYP	0	1	0	0
10	untuk	9	RESERVED	NOTYP	0	1	0	0
11	fungsi	10	RESERVED	NOTYP	0	1	0	0
12	jika	11	RESERVED	NOTYP	0	1	0	0
13	mod	12	RESERVED	NOTYP	0	1	0	0
14	tidak	13	RESERVED	NOTYP	0	1	0	0
15	dari	14	RESERVED	NOTYP	0	1	0	0
16	atau	15	RESERVED	NOTYP	0	1	0	0
17	prosedur	16	RESERVED	NOTYP	0	1	0	0
18	program	17	RESERVED	NOTYP	0	1	0	0
19	record	18	RESERVED	NOTYP	0	1	0	0
20	ulangi	19	RESERVED	NOTYP	0	1	0	0
21	string	20	RESERVED	NOTYP	0	1	0	0
22	maka	21	RESERVED	NOTYP	0	1	0	0
23	ke	22	RESERVED	NOTYP	0	1	0	0
24	tipe	23	RESERVED	NOTYP	0	1	0	0
25	sampai	24	RESERVED	NOTYP	0	1	0	0
26	variabel	25	RESERVED	NOTYP	0	1	0	0
27	selama	26	RESERVED	NOTYP	0	1	0	0
28	packed	27	RESERVED	NOTYP	0	1	0	0
29	integer	-1	TYPE	INT	0	1	0	1

30	real	29	TYPE	REAL	0	1	0	1
31	boolean	30	TYPE	BOOL	0	1	0	1
32	char	31	TYPE	CHAR	0	1	0	1
33	true	32	CONST	BOOL	1	1	0	1
34	false	33	CONST	BOOL	0	1	0	0
35	read	34	PROC	NOTYP	0	1	0	0
36	readln	35	PROC	NOTYP	0	1	0	1
37	write	36	PROC	NOTYP	0	1	0	2
38	writeln	37	PROC	NOTYP	0	1	0	3
39	TestConstants	38	PROGRAM	NOTYP	0	1	0	0
40	MAX_SIZE	39	CONST	INT	0	1	0	100
41	MIN_VALUE	40	CONST	INT	0	1	0	0
42	PI	41	CONST	REAL	0	1	0	0
43	GREETING	42	CONST	NOTYP	0	1	0	0
44	FLAG	43	CONST	BOOL	0	1	0	1
45	LETTER	44	CONST	CHAR	0	1	0	65
46	size	45	VAR	INT	0	1	0	0
47	value	46	VAR	REAL	0	1	0	1
48	flag	47	VAR	BOOL	0	1	0	2
49	ch	48	VAR	CHAR	0	1	0	3

=== BTAB (Block Table) ===

Idx	Last	Lpar	Psize	Vsize
0	49	0	0	4
1	49	0	0	0

DECORATED AST:

ProgramNode(name: 'TestConstants', tab_index: 39)

Declarations:

ConstDecl(name: 'MAX_SIZE', type: integer, tab_index: 40)

Value:

Number(int: 100, type: integer)

ConstDecl(name: 'MIN_VALUE', type: integer, tab_index: 41)

Value:

Number(int: 0, type: integer)

ConstDecl(name: 'PI', type: real, tab_index: 42)

Value:

Number(real: 3.14, type: real)

ConstDecl(name: 'GREETING', type: string, tab_index: 43)

Value:

String(value: "Hello")

ConstDecl(name: 'FLAG', type: boolean, tab_index: 44)

Value:

Bool(value: true)

ConstDecl(name: 'LETTER', type: char, tab_index: 45)

Value:

Char(value: 'A')

VarDecl(name: 'size', type: integer, tab_index: 46, level: 0)

VarDecl(name: 'value', type: real, tab_index: 47, level: 0)

VarDecl(name: 'flag', type: boolean, tab_index: 48, level: 0)

VarDecl(name: 'ch', type: char, tab_index: 49, level: 0)

Body:

CompoundStatement

Assign

```

    Target:
      Var(name: 'size', tab_index: 46, type: integer)
    Value:
      Var(name: 'MAX_SIZE', tab_index: 40, type: integer)
Assign
  Target:
    Var(name: 'value', tab_index: 47, type: real)
  Value:
    BinOp(op: '*', type: real)
      Left:
        Var(name: 'PI', tab_index: 42, type: real)
      Right:
        Number(int: 2, type: integer)
Assign
  Target:
    Var(name: 'flag', tab_index: 48, type: boolean)
  Value:
    Var(name: 'FLAG', tab_index: 44, type: boolean)
Assign
  Target:
    Var(name: 'ch', tab_index: 49, type: char)
  Value:
    Var(name: 'LETTER', tab_index: 45, type: char)
Assign
  Target:
    Var(name: 'size', tab_index: 46, type: integer)
  Value:
    BinOp(op: '-', type: integer)
      Left:
        Var(name: 'MAX_SIZE', tab_index: 40, type: integer)
      Right:
        Var(name: 'MIN_VALUE', tab_index: 41, type: integer)
Assign
  Target:
    Var(name: 'value', tab_index: 47, type: real)
  Value:
    BinOp(op: '+', type: real)
      Left:
        Var(name: 'PI', tab_index: 42, type: real)
      Right:
        Number(real: 1.5, type: real)
If
  Condition:
    BinOp(op: '>', type: boolean)
      Left:
        Var(name: 'size', tab_index: 46, type: integer)
      Right:
        Var(name: 'MAX_SIZE', tab_index: 40, type: integer)
  For(variable: 'size', direction: to)
    Start:
      Var(name: 'MIN_VALUE', tab_index: 41, type: integer)
    End:
      Var(name: 'MAX_SIZE', tab_index: 40, type: integer)
    Body:
      Assign
        Target:

```

```
    Var(name: 'value', tab_index: 47, type: real)
Value:
  BinOp(op: '+', type: real)
    Left:
      Var(name: 'value', tab_index: 47, type: real)
    Right:
      Number(int: 1, type: integer)
```

Bukti

```
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-3/test_constants.pas
-----
Starting syntax analysis...
-----
Parse Tree:
-----
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestConstants)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <const-declaration>
    │   ├── KEYWORD(konstanta)
    │   ├── IDENTIFIER(MAX_SIZE)
    │   ├── RELATIONAL_OPERATOR(=)
    │   ├── <constant-value>(100)
    │   ├── SEMICOLON(;)
    │   ├── IDENTIFIER(MIN_VALUE)
    │   ├── RELATIONAL_OPERATOR(=)
    │   ├── <constant-value>(0)
    │   ├── SEMICOLON(;)
    │   ├── IDENTIFIER(PI)
    │   └── RELATIONAL_OPERATOR(=)
```

```

        Var(name: 'PI', tab_index: 42, type: real)
    Right:
        Number(real: 1.5, type: real)
If
    Condition:
        BinOp(op: '>', type: boolean)
    Left:
        Var(name: 'size', tab_index: 46, type: integer)
    Right:
        Var(name: 'MAX_SIZE', tab_index: 40, type: integer)
For(variable: 'size', direction: to)
    Start:
        Var(name: 'MIN_VALUE', tab_index: 41, type: integer)
    End:
        Var(name: 'MAX_SIZE', tab_index: 40, type: integer)
    Body:
        Assign
        Target:
            Var(name: 'value', tab_index: 47, type: real)
        Value:
            BinOp(op: '+', type: real)
            Left:
                Var(name: 'value', tab_index: 47, type: real)
            Right:
                Number(int: 1, type: integer)
-----
Semantic analysis completed!
-----

```

d. error_type_redeclaration.pas

Input

```

<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestTypeRedeclaration)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <type-declaration>
    │   ├── KEYWORD(tipe)
    │   ├── <type-definition>
    │   │   ├── IDENTIFIER(Counter)
    │   │   ├── RELATIONAL_OPERATOR(=)
    │   │   ├── <type>
    │   │   │   └── KEYWORD(integer)
    │   │   └── SEMICOLON(;)
    │   └── <type-definition>
    │       └── IDENTIFIER(Index)
    
```

```

      RELATIONAL_OPERATOR(=)
      <type>
      | KEYWORD(integer)
      | SEMICOLON(;)
    <type-definition>
    | IDENTIFIER(Counter)
    | RELATIONAL_OPERATOR(=)
    | <type>
    | | KEYWORD(real)
    | | SEMICOLON(;)
  <var-declaration>
  | KEYWORD(variabel)
  | <identifier-list>
  | | IDENTIFIER(c)
  | COLON(:)
  | <type>
  | | <custom-type>(Counter)
  | SEMICOLON(;)
  | <identifier-list>
  | | IDENTIFIER(idx)
  | COLON(:)
  | <type>
  | | <custom-type>(Index)
  | SEMICOLON(;)
<compound-statement>
| KEYWORD(mulai)
| <statement-list>
| | <assignment-statement>
| | | IDENTIFIER(c)
| | | ASSIGN_OPERATOR(:=)
| | | <expression>
| | | | <simple-expression>
| | | | | <term>
| | | | | <factor>
| | | | | NUMBER(10)
| | SEMICOLON(;)
| | <assignment-statement>
| | | IDENTIFIER(idx)
| | | ASSIGN_OPERATOR(:=)
| | | <expression>
| | | | <simple-expression>
| | | | | <term>
| | | | | <factor>
| | | | | NUMBER(1)
| KEYWORD(selesai)
DOT(.)

```

Output

Error: Semantic Error at line 8: Type 'Counter' already declared in this scope

Bukti


```
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-3/error_type_redeclaration.pas
```

```
-----
Starting syntax analysis...
-----
```

```
Parse Tree:
```

```
-----
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestTypeRedeclaration)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <type-declaration>
    │   ├── KEYWORD(tipe)
    │   ├── <type-definition>
    │   │   ├── IDENTIFIER(Counter)
    │   │   ├── RELATIONAL_OPERATOR(=)
    │   │   ├── <type>
    │   │   │   └── KEYWORD(integer)
    │   └── SEMICOLON(;)
    └── <type-definition>
```

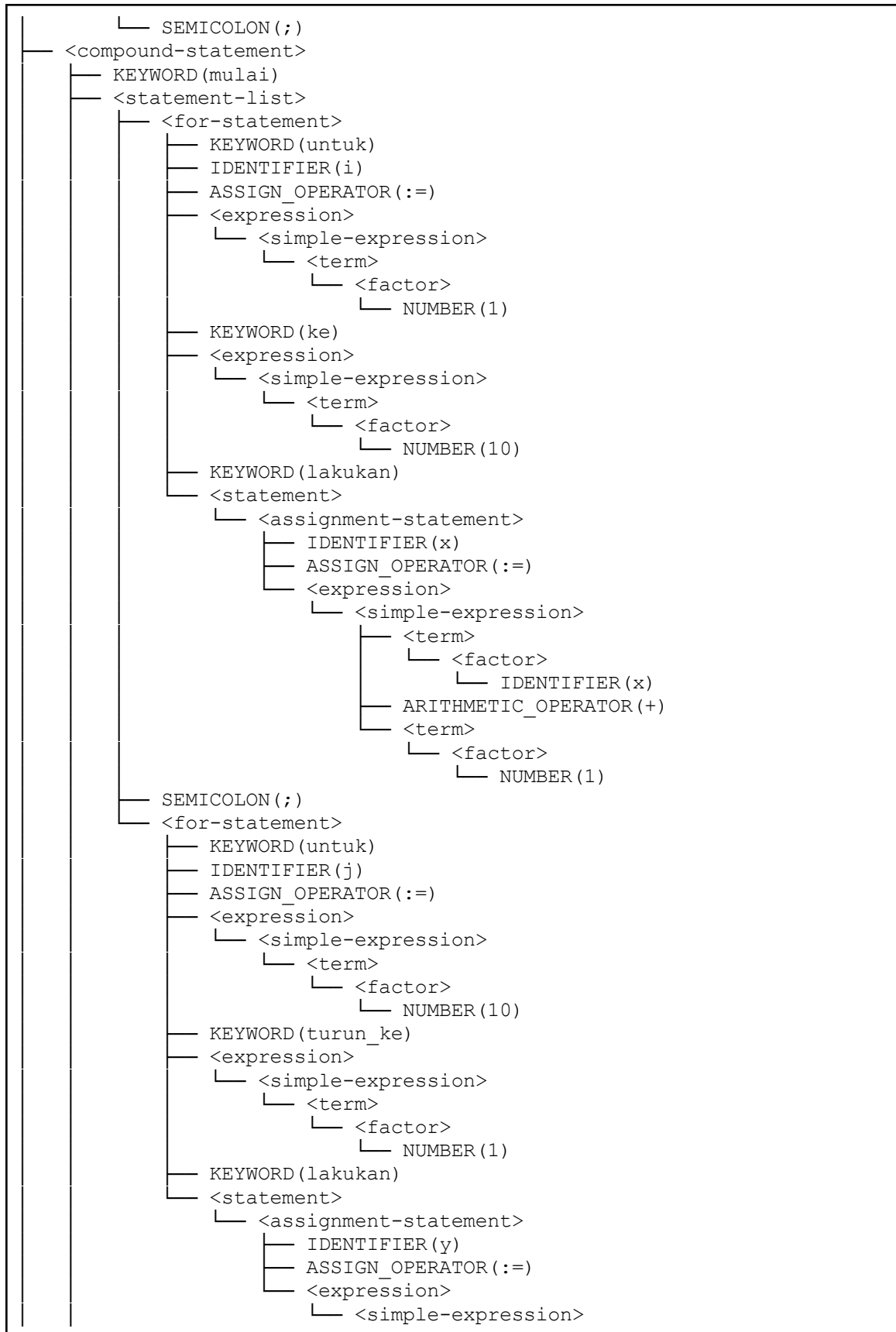
```
-----
SEMANTIC ERRORS:
```

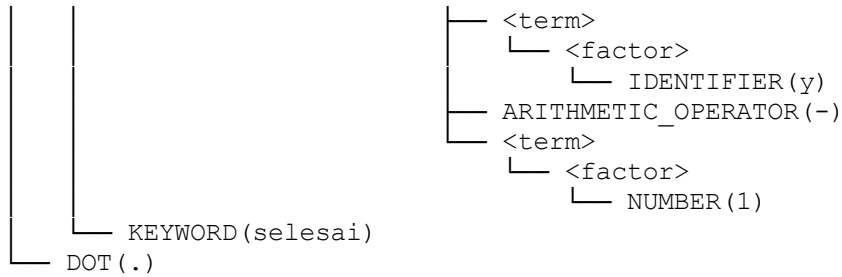
```
-----
Error: Semantic Error at line 8: Type 'Counter' already declared in this scope
-----
```

e. error_for_undeclared.pas

Input

```
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestForUndeclared)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <var-declaration>
    │   ├── KEYWORD(variabel)
    │   ├── <identifier-list>
    │   │   ├── IDENTIFIER(x)
    │   │   ├── COMMA(,)
    │   │   └── IDENTIFIER(y)
    │   ├── COLON(:)
    │   ├── <type>
    │   │   └── KEYWORD(integer)
    └──
```





Output

Error: Semantic Error: Loop variable 'i' not declared
Error: Semantic Error: Loop variable 'j' not declared

Bukti

```
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: test/milestone-3/error_for_undeclared.pas
-----
Starting syntax analysis...
-----
Parse Tree:
-----
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestForUndeclared)
│   └── SEMICOLON(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── KEYWORD(variabel)
│       ├── <identifier-list>
│       │   ├── IDENTIFIER(x)
│       │   ├── COMMA(,)
│       │   └── IDENTIFIER(y)
│       ├── COLON(:)
│       ├── <type>
│       │   └── KEYWORD(integer)
│       └── SEMICOLON(;)
└── <compound-statement>
```

```
-----  
SEMANTIC ERRORS:  
-----
```

```
Error: Semantic Error: Loop variable 'i' not declared  
Error: Semantic Error: Loop variable 'j' not declared  
-----
```

4. Kesimpulan & Saran

a. Kesimpulan

Implementasi semantic analyzer untuk Milestone 1 Tugas Besar IF3170 telah berhasil diselesaikan dengan memanfaatkan konsep Syntax-Directed Translation Scheme dan L-Attributed Grammar. Semantic analyzer yang dibangun mampu menjalankan fungsinya, yaitu menghasilkan sebuah abstract syntax tree dari parse tree di tahap sebelumnya dan melakukan analisis semantik menggunakan aturan-aturan produksi dan aksi semantik yang ada.

Berdasarkan hasil pengujian terhadap berbagai file uji, semantic analyzer telah mampu untuk:

- Mengonversi parse tree menjadi Abstract Syntax Tree (AST) yang lebih sederhana dan cocok untuk analisis semantik, sambil mempertahankan informasi posisi (nomor baris) pada node penting.
- Membangun dan memelihara Symbol Table (TAB/BTAB/ATAB): menambah entri deklarasi, mengelola enter/exit scope, serta menyimpan metadata tipe dan alamat/offset variabel dan parameter.
- Mendeteksi dan melaporkan kesalahan deklarasi seperti undeclared identifier dan redeclaration dalam scope yang sama, dengan pesan yang terlokalisir menggunakan nomor baris.
- Melakukan pemeriksaan tipe pada assignment, ekspresi, dan pemanggilan subprogram (type checking), termasuk validasi operator aritmatika, logika, dan relasional, serta inferensi tipe hasil operasi (integer/real/boolean).
- Memverifikasi kesesuaian pemanggilan prosedur/fungsi (ekspektasi jumlah dan tipe argumen) dan menangani dekorasi node fungsi dengan tipe kembalian.
- Menangani deklarasi tipe kompleks (array, record, subrange), menambah meta data array ke ATAB, serta menghitung ukuran dan alamat untuk variabel/parameter.
- Menghitung alamat/offset parameter dan variabel lokal per blok (BTAB) untuk persiapan code generation.

- Membangun decorated AST: setiap node yang relevan diberi anotasi tipe, indeks simbol, dan level scope sehingga output dapat dipakai untuk debugging atau tahap lanjutan.
- Mengumpulkan dan menyajikan daftar error semantik yang informatif.

Secara keseluruhan, semantic analyzer yang diimplementasikan telah fungsional dan memenuhi spesifikasi yang dibutuhkan untuk milestone ini. Semantic analyzer ini dapat digunakan lebih lanjut ke tahap-tahap selanjutnya dalam mengonstruksi sebuah compiler.

b. Saran

Semantic analyzer sudah dapat bekerja dengan baik. Namun, ada beberapa penambahan atau perbaikan yang dapat diimplementasikan. Misalnya, modularisasi kode yang lebih baik supaya satu file tidak berisi terlalu banyak baris.

Lampiran

a. Link Repository Github:

[Darsua/YRH-Tubes-IF2224: NieR: Finite Automata](#)

b. Pembagian Tugas

Anggota	Tugas	Kontribusi
Muhammad Ra'if Alkautsar (13523011)	Parser	35%
	Laporan	20%
	Main	50%
Fajar Kurniawan (13523027)	Parser	20%
	Laporan	40%
Darrel Adinarya Sunanda (13523061)	Laporan	10%
	Main	50%
Reza Ahmad Syarif (13523119)	Parser	45%
	Laporan	30%

Referensi

“PASCAL-S: A Subset and its implementation”

<http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf>

“Let’s Build A Simple Interpreter. Part 7: Abstract Syntax Trees”

<https://ruslanspivak.com/lbasi-part7>

Semantic Analysis in Compiler Design

https://www.tutorialspoint.com/compiler_design/compiler_design_semantic_analysis.htm

“Contoh Gambar Parse Trees”

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png